

# The Current Challenges of Software Engineering in the Era of Large Language Models

CUIYUN GAO, Harbin Institute of Technology Shenzhen, Shenzhen, China

XING HU, Zhejiang University, Hangzhou, China

SHAN GAO, Huawei Technologies Co Ltd China, Hangzhou, China

XIN XIA, Software Engineering Application Lab, Huawei Technologies Co Ltd, Hangzhou, China

ZHI JIN, Peking University, Beijing, China

With the advent of large language models (LLMs) in the AI area, the field of software engineering (SE) has also witnessed a paradigm shift. These models, by leveraging the power of deep learning and massive amounts of data, have demonstrated an unprecedented capacity to understand, generate, and operate programming languages. They can assist developers in completing a broad spectrum of software development activities, encompassing software design, automated programming, and maintenance, which potentially reduces huge human efforts. Integrating LLMs within the SE landscape (LLM4SE) has become a burgeoning trend, necessitating exploring this emergent landscape's challenges and opportunities.

The article aims at revisiting the software development lifecycle (SDLC) under LLMs, and highlighting challenges and opportunities of the new paradigm. The article first summarizes the overall process of LLM4SE, and then elaborates on the current challenges based on a thorough discussion. The discussion was held among more than 20 participants from academia and industry, specializing in fields such as SE and artificial intelligence. Specifically, we achieve 26 key challenges from seven aspects, including software requirement and design, coding assistance, testing code generation, code review, code maintenance, software vulnerability management, and data, training, and evaluation. We hope the achieved challenges would benefit future research in the LLM4SE field.

CCS Concepts: • **Software and its engineering** → **Software development techniques**;

Additional Key Words and Phrases: Large Language Models, Challenges, LLM4SE

## ACM Reference format:

Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. 2025. The Current Challenges of Software Engineering in the Era of Large Language Models. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 127 (May 2025), 30 pages. <https://doi.org/10.1145/3712005>

This research is supported by National Natural Science Foundation of China (Project Nos. 62472126, 62192731, and 62192730), Natural Science Foundation of Guangdong Province (Project No. 2023A1515011959), Shenzhen-Hong Kong Jointly Funded Project (Category A, No. SGDX20230116091246007), Shenzhen Basic Research (General Project No. JCYJ20220531095214031), Shenzhen International Science and Technology Cooperation Project (No. GJHZ20220913143008015), and CCF-Huawei Populus Grove Fund.

Authors' Contact Information: Cuiyun Gao, Harbin Institute of Technology Shenzhen, Shenzhen, China; e-mail: gaocuiyun@hit.edu.cn; Xing Hu (corresponding author), Zhejiang University, Hangzhou, China; e-mail: xinghu@zju.edu.cn; Shan Gao, Huawei Technologies Co Ltd China, Hangzhou, China; e-mail: gaoshan17@huawei.com; Xin Xia, Software Engineering Application Lab, Huawei Technologies Co Ltd, Hangzhou, China; e-mail: xin.xia@acm.org; Zhi Jin (corresponding author), Peking University, Beijing, China; e-mail: zhijin@pku.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2025/5-ART127

<https://doi.org/10.1145/3712005>

## 1 Introduction

In the 1960s, **software engineering (SE)** was formally established as a discipline in response to software's increasing scale and complexity [126], based on the symposium on SE organized by the North Atlantic Treaty Organization. In the nascent stage of SE, during which AI techniques were not widely adopted, the field was more focused on structured programming [94], modular design [136], and data structure [59], which greatly promoted the widespread application of programming languages and compiler systems. Along with the explosive development of object-oriented programming and cloud computing, and continuing maturity of the AI techniques, the field of intelligent SE (also called *AI4SE*) [56] has gradually emerged. In the field, AI techniques have been introduced into various stages of the **software development lifecycle (SDLC)**, including programming, testing, and maintenance, aiming at boosting the efficiency and quality of software development. Nowadays, AI4SE is still playing an increasingly essential role in a growing number of SE domains, including but not limited to intelligent requirements analysis and design [20, 118], automated code generation and repair [70, 90, 109], as well as intelligent project management [7, 93]. According to Gartner's report for 2023 [50], artificial intelligence-augmented SE is listed as one of the top strategic technology trends in SE, indicating the general trend of enhancing SE with AI techniques.

Recently, the rapid advancement of the Artificial Intelligence Generated Content technology [22], especially **large language models (LLMs)** [42, 181], is ushering in new challenges and opportunities for the SE field, and displays potential for transforming the software development paradigm. LLM-based software engineering (also called *LLM4SE*) has attracted widespread attention from both academia and industry. The practical application of tools such as GitLab Duo [37] and GitHub Copilot X [29] have demonstrated the broad prospects of LLM4SE, inspiring new software development mode. Specifically, by breaking through the traditional multi-stage software development process, LLM4SE would make an end-to-end software development possible, i.e., generating and testing code directly based on given requirements.

The prospects of LLM4SE have acted as the "catfish effect," promoting the exploration of new software development modes among academia and industry. However, due to limitations such as the essential semantic gap between code and natural languages, constantly updated data and models, and instability of generated results, existing LLM4SE techniques are still faced with serious challenges regarding the quality, efficiency, reliability, and trustworthiness of developed software. The challenges have also sparked a new wave of enthusiasm for SE technologies. To better understand the issues in the current SE stage, 24 academia researchers and industry practitioners, specializing in different fields such as SE and artificial intelligence, were grouped to discuss the opportunities and challenges in the era of LLM4SE from 19 January 2024 to 21 January 2024 at the "9th CCF Beautiful Lake Seminars" [1]. Based on the rigorous discussion, the article summarizes 26 key challenges from seven aspects, including software requirement and design, coding assistance, testing code generation, code review, software maintenance, software vulnerability management, and data, training, and engineering. This article aims to provide insights for researchers and practitioners to effectively harness LLMs' strengths while mitigating their weaknesses, thereby applying them to support continuous and robust software development.

The major contributions of this article are summarized as follows:

- (1) We perform a qualitative study to investigate and summarize the current challenges of LLM4SE.
- (2) We achieve findings from various aspects in LLM4SE, and provide practical implications for researchers and practitioners to inspire future avenues of research.

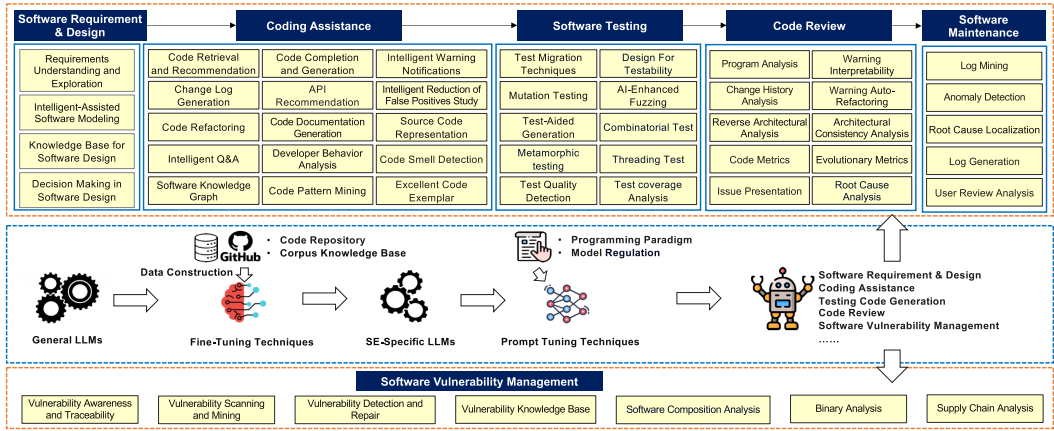


Fig. 1. Overall process of LLM4SE.

The remainder of this article is structured as follows: Section 2 provides the background of our work, including the overall process of LLM4SE and common techniques in each step. Section 3 describes our methodology for investigating the challenges in LLM4SE. Sections 4 and 5 present the summarized challenges and related work, respectively. Section 6 discusses the threats to the validity of our work, and Section 7 concludes the article.

## 2 Background

In this section, we first illustrate the overall process of LLM4SE, and then elaborate on the details in each step including the data construction, fine-tuning techniques, SE-specific LLMs, prompt tuning techniques, and downstream tasks.

### 2.1 Overall Process of LLM4SE

Figure 1 presents the overall process of the integration of LLM into SE applications, including data construction, fine-tuning, prompting techniques, and SE-specific LLMs. With the support of SE-specific LLMs, multiple SE tasks (e.g., design, coding, testing, reviewing, maintenance, and vulnerability management) can be enhanced substantially.

### 2.2 Data Construction

The remarkable performance of the previously mentioned large code models can be attributed to the extensive dataset and the meticulously structured data organization employed during the model's pre-training phase.

**2.2.1 Data Collection.** Data collection is the basic step for constructing a high-quality corpus of source code. A common solution is to collect publicly available source code data from GitHub or public archive [98]. For instance, Guo et al. [54] crawl public repositories created before February 2023 on GitHub with 87 programming languages. The StarCoder2 team collects the source code from the Software Heritage archive [3]. Then the authors start by extracting the most recently crawled versions of all GitHub repositories and filtering them to retain only the main branch.

**2.2.2 Data Filtering.** Source code data from public repositories may not always be clean or useful, and malicious data may harm model performance and leak confidential user data. Therefore, delicate data filtering is also crucial for powerful large code models. According to Lozhkov et al. [98], many filtering rules are applied for StarCoder2, including (1) *Long line filters* to eliminate too

long code lines, (2) *Auto-generated filter* to remove auto-generated files such as compilation files, (3) *Alpha filter* to remove files with less than 25% of alphabetic characters, and (4) *Encoded data filter* to remove content about encoding data.

**2.2.3 Data Organization.** According to the work [54], there exists an inter-relationship among the files of source code. Thus, the authors propose a Topological Sort for Dependency Analysis algorithm for organizing code files. Based on the dependencies, models are trained with the files in a certain order to learn such dependencies. For StarCoder2 [98], authors utilize different special tokens such as `< commit >` to represent different data types, such as commit messages.

## 2.3 Fine-Tuning Techniques

Through pre-training, LLMs learn a huge amount of knowledge about SE and source code. However, LLMs are not aligned with human preference, resulting in their limited capabilities to serve as intelligent assistants. Therefore, to better adapt LLMs for software development practice, researchers primarily focus on four tuning techniques: adapter-based tuning, **low-rank adaptation (LoRA)**, prefix tuning, and prompt tuning.

**2.3.1 Adapter-Based Tuning.** Adapter-based tuning introduces adapter modules with a bottleneck structure between the layers of pre-trained models. In the fine-tuning stage, the parameters of the original pretrained models are fixed, and only adapter modules are adjusted. In particular, Wang et al. [152] introduce adapter tuning to improve code search and summarization by adding a small number of trainable parameters to pre-trained models. Their approach reduces catastrophic forgetting in LLM and achieves superior performance in cross-lingual and low-resource scenarios.

**2.3.2 LoRA.** LoRA injects trainable rank decomposition matrices into pretrained models to substitute the original weights, which can amplify some hidden features encoded during pre-training. In the study by Liu et al. [95], they explore how LoRA modifies pre-trained models for code-change tasks by injecting low-rank matrices that substitute original weights. Through this method, they effectively capture dynamic code changes while reducing the need for extensive retraining.

**2.3.3 Prefix Tuning.** Prefix tuning introduces a set of prefix parameters to the beginning of each layer in the LLM. These parameters serve as adjustments that precede the original input during processing. LLaMA-Adapter [188] incorporates a set of trainable adaptation embeddings and attaches them to the word embeddings in the upper layers of the LLMs. A zero-initialized attention scheme with zero gating is also introduced. It dynamically incorporates new guiding signals into LLaMA while retaining its pretrained knowledge.

**2.3.4 Prompt Tuning.** Prompt tuning modifies an LLM's input by incorporating a natural language prompt designed to predict masked tokens. This adjustment maintains the input format consistent with that used during the model's pre-training, facilitating more effective task-specific adjustments. For instance, Wang et al. [149] apply prompt tuning to pre-trained models like CodeBERT and CodeT5 and showing its effectiveness over fine-tuning in various code intelligence tasks at low-resource scenario. The inserted natural language prompt can involve task-specific knowledge to facilitate the adaption to downstream tasks.

## 2.4 SE-Specific LLMs

At the end of 2021, OpenAI released CodeX [25] as the first LLM designed explicitly for code generation. With a similar architecture to GPT-3 [19], CodeX was pre-trained using 159 GB of public code data from GitHub. Based on CodeX, the Copilot plugin [29] has become one of the most

popular tools for code generation. Additionally, the HumanEval dataset proposed in the work [25] has become one of the standard evaluation datasets for subsequent code generation tasks. In 2022, DeepMind proposed AlphaCode [88], a model that employed an encoder-decoder architecture and was pre-trained on 715.1 GB of code data across 12 programming languages. It surpassed 54.3% of human programmers in a CodeForce competition with 5,000 participants.

In recent years, a growing number of open source SE-Specific LLMs have been proposed. In 2020, Microsoft proposed CodeGPT [138], based on the architecture of GPT-2, which was used to support the embedded code completion tool of Visual Studio. Wang et al. proposed CodeT5 [156] that follows an encoder-decoder architecture to support various code intelligence tasks, including code translation and code summarization. Salesforce proposed CodeGEN [114], an open source model with 1.61 billion parameters, pre-trained on over 800 GB of data. Unlike other models that generate code in a single step, CodeGEN facilitates conversational program generation. The process of code generation with the help of the model can be seen as a multi-round dialogue between the user and the model, where the user provides requirements for the code generation model in multiple instances and receives several outputs from the model.

In addition, Meta AI proposed CodeLLaMA, a family of code LLMs based on LLaMA 2 [127] with the capabilities of code generation, blank infilling, and long-context processing. The BigCode project introduced StarCoder [86], an LLM trained on the mixture of source code and natural language texts. Its training data incorporate more than 80 different programming languages, as well as text extracted from GitHub issues and commits and from notebooks. The total account of training tokens exceeds 1T. Based on CodeLLaMA, Guo et al. proposed DeepSeek-Coder [54], which is a series of code LLMs that have an identical architecture to CodeLLaMA. DeepSeek-Coder is trained from 2T tokens from scratch, which comprises 87% code and 13% natural language in both English and Chinese. DeepSeek-Coder has achieved state-of-the-art performance in various code intelligence tasks, e.g., code generation and code completion. Lozhkov et al. proposed StarCoder2 [98] and a larger-scale source code dataset named The Stack V2. Based on purely open sourced data, StarCoder2 achieves state-of-the-art effectiveness in math and code reasoning benchmarks, as well as several low-resource languages. The aiXcoder team has launched aiXcoder-7B [71], a lightweight and effective LLM for code completion. More LLMs for SE are illustrated in Table 1.

## 2.5 Prompting Techniques

Prompting techniques are the key factors in adapting LLMs into SE practice. In this section, we categorize the prompting techniques into two types including training-time and test-time techniques.

*Training-Time Techniques.* Wang et al. [149, 150] observe that prompt tuning that is equipped with the templates describing task formulation and specification can further boost the performance in downstream tasks compared to fine-tuning. Luo et al. [104] utilize the prompting techniques Evol-Instruct [170] to construct a more complex dataset and adopt a delicately designed template to describe the task in the training process.

*Test-Time Techniques.* A direct way to use LLMs in SE tasks is *zero-shot prompting*, which describes the task and obtains answers from LLMs. To make LLMs further understand the task, *few-shot prompting* is proposed [19] (is also called in-context prompting). Before asking the question, multiple input-output pairs are selected as examples to present to LLMs. With the demonstrated examples, LLMs can further understand the task specification and output format, producing better performance [49]. In addition, to further specify the task and improve the reasoning capabilities of LLMs, Wei et al. propose **chain-of-thought (CoT)** [158] that prompts LLMs the detailed reasoning process instead of directly showing the answer, substantially improving the performance of LLMs.

Table 1. LLMs for SE

| Model                  | Developer           | Release Date | Parameter Quantity (B) | Pre-Trained Data Size    | Architecture    |
|------------------------|---------------------|--------------|------------------------|--------------------------|-----------------|
| CodeX [25]             | OpenAI              | 2021.07      | 2.5/12                 | 100B tokens              | Decoder         |
| AlphaCode [88]         | DeepMind            | 2022.02      | 41                     | 967B tokens              | Encoder-decoder |
| CodeGen [114]          | Salesforce          | 2022.03      | 2.7/6.1/16             | 577B tokens              | Decoder         |
| AiXcoder L             | AiXcoder            | 2022.04      | -                      | -                        | Decoder         |
| AiXcoder XL [55]       | AiXcoder            | 2022.06      | -                      | -                        | Decoder         |
| PanGu-Coder [28]       | HUAWEI              | 2022.07      | 2.6                    | 147 GB                   | Decoder         |
| CodeGeeX [191]         | Zhipu.AI            | 2022.09      | 13                     | 850B tokens              | Decoder         |
| BLOOM [128]            | BigScience          | 2022.09      | 176                    | 1.61 TB                  | Decoder         |
| PaLM-Coder [27]        | Google              | 2022.10      | 8/62/540               | 780B tokens              | Decoder         |
| SantaCoder [9]         | Huggingface         | 2023.01      | 1.1                    | 118B tokens              | Decoder         |
| InCoder [46]           | Facebook            | 2023.04      | 1.3/6.7                | 216 GB                   | Decoder         |
| StarCoder [86]         | Huggingface         | 2023.05      | 15.5                   | 1T tokens                | Decoder         |
| CodeGen2 [113]         | Salesforce          | 2023.05      | 3.7/7/16               | 400B tokens              | Decoder         |
| CodeT5+ [154]          | Salesforce          | 2023.05      | 2/6/16                 | 51.1B tokens             | Encoder-decoder |
| WizardCoder [104]      | Microsoft           | 2023.06      | 15                     | -                        | Decoder         |
| Phi-1 [52]             | Microsoft           | 2023.06      | 13                     | 7B tokens                | Decoder         |
| PanGu-Coder2 [129]     | HUAWEI              | 2023.07      | 15                     | -                        | Decoder         |
| CodeLLaMa [127]        | Meta                | 2023.08      | 7/13/34                | 116K tokens              | Decoder         |
| CodeQwen [12]          | Alibaba             | 2023.09      | 7/14                   | 90B tokens               | Decoder         |
| Phi-1.5 [87]           | Microsoft           | 2023.09      | 1.3                    | 27B tokens               | Decoder         |
| DeepSeek-Coder [54]    | DeepSeek AI         | 2023.11      | 1/5.7/6.7/33           | 2T tokens                | Decoder         |
| Phi-2                  | Microsoft           | 2023.12      | 2.7                    | 1.4T tokens              | Decoder         |
| AlphaCode2             | DeepMind            | 2023.12      | -                      | -                        | Decoder         |
| Magicoder [159]        | UIUC and THU        | 2023.12      | 7                      | -                        | Decoder         |
| WaveCoder [179]        | Microsoft           | 2024.01      | 15                     | -                        | Decoder         |
| StarCoder2 [98]        | Huggingface         | 2024.01      | 3/7/15                 | 622B+/658B+/913B+ tokens | Decoder         |
| CodeQwen1.5            | Alibaba             | 2024.04      | 7                      | 3T tokens                | Decoder         |
| SEMCODER [33]          | Columbia University | 2024.06      | 6.7                    | -                        | Decoder         |
| DeepSeek-Coder-V2 [30] | DeepSeek AI         | 2024.06      | 16/236                 | 10.2T tokens             | MoE             |
| Qwen2.5-Coder [173]    | Alibaba             | 2024.09      | 1.5/7/32               | 5.5T tokens              | Decoder         |

The symbol “-” indicates the corresponding value is not publicly accessible.

Equipped with CoT, LLMs have also been proven to present a better performance in SE tasks such as code generation [54].

2.6 Downstream Tasks

The LLM4SE workflow can be applied to various SE tasks, including software requirement and design, coding assistance, testing code generation, code review, software maintenance, and software vulnerability management. Specifically, for the software requirement and design task, LLMs can serve as a knowledge base for software design, or conduct code-centric software design, and so on. For the coding assistance, LLMs are beneficial for a series of tasks, including code retrieval and recommendation, code pattern mining, code completion and generation, and intelligent Q&A. For testing code generation, LLMs could support threading test, test coverage analysis, and AI-enhanced fuzzing. For the software maintenance, LLMs are able to support many tasks such as log mining, anomaly detection, and user review analysis. Finally, for the software vulnerability management, LLMs can facilitate vulnerability awareness and traceability, binary analysis, and supply chain analysis.

3 Methodology

To investigate the challenges in LLM4SE, 24 participants with experience in software development had face-to-face meetings in the “9th CCF Beautiful Lake Seminars” during 19 January 2014 to 21 January 2024. The participants include 17 academic researchers and 7 industry practitioners, specializing in different fields such as SE and machine learning. Six thematic sessions were performed, namely “Software Engineering under Large Models,” “Large Models for Code Intelligence,” “Large



Models for Software Quality Assurance,” “Data, Evaluation, and Validation for Large Code Models,” “Large Models for Open-Source Engineering,” and “Future Trends and Challenges of LLM4SE,” each followed by a panel or collective discussion, with detailed topics illustrated in the public repository.<sup>1</sup> Each thematic seminar together with the discussion lasted for around 4 hours for ensuring the corresponding topics were thoroughly discussed. We performed the following coding procedures to summarize the discussed challenges.

(1) *Transcribing and coding*. Following the procedure in the work [62, 148], the first author transcribes the discussion of each seminar and uses NVivo [38] to conduct open coding and generate opinion cards. Then another author verifies the summarized opinion cards and provides suggestions for improvement.

(2) *Open card sorting*. The two authors then separately grouped the codes into potential challenges for thematic similarity according to the study by LaToza et al. [79]. The two authors discuss their disagreements to reach a common decision. To reduce the bias of the two authors in sorting descriptions, another two authors have also reviewed and confirmed the final set of challenges. Finally, we derived 26 key challenges corresponding to seven aspects, with details illustrated in Section 4.

## 4 Current Challenges in LLM4SE

In this section, we present the challenges of LLM4SE. We introduce them from six aspects, including the challenges in the SE process, e.g., software requirement and design, coding assistance, testing, code analysis and review, software vulnerability management, as well as the challenges relevant to the construction of SE-specific LLMs, e.g., challenges in data, training, and evaluation.

### 4.1 Challenges in Requirement and Design

Requirement [115] and design [176] constitute critical phases in the software development process, and serve as initial stages of the process. The primary challenges stem from two main aspects, including the inherent complexity of real-world problems and communication barriers between domain experts and software developers. These two aspects lead to increased uncertainty in defining requirements during the early stages of development, and thereby unexpected design. The powerful capabilities of LLMs in natural language processing can effectively mitigate expression uncertainties, thereby beneficial for addressing the challenges encountered during the requirements and design stages. However, how to ensure that LLMs can effectively capture the intricate requirements and design in the complex software still remains challenging.

(1) *Requirement/design prompts*. LLMs heavily rely on comprehensive prompts and available contextual information to generate effective outputs. Even slightly different prompts can lead to significantly different results [149]. Besides, it is hard to render the prompts anticipate the costs and risk of the generated requirements/design [166]. Therefore, when employing LLMs as requirement and design agents, it is essential to conduct thorough empirical evaluations on the prompts.

(2) *Structured descriptions of the communication between domain experts and software developers*. LLMs present inherent limitations in context length and inductive bias (influenced by training data) [82]. For instance, the context length of ChatGPT (e.g., gpt-3.5-turbo) is restricted to 16,385 tokens, which is usually sufficient but still hard to process lengthy requirement documents or maintain the task context over extended conversations.

(3) *Lack of domain expert knowledge*. Requirements engineers are generally required to comprehend the application domain for accurate and complete requirement extraction. However, LLMs may have limited training on specific domain knowledge, necessitating the integration of domain

<sup>1</sup><https://docs.google.com/spreadsheets/d/1ewVENYYq1UyKBQDaAgC8-45WMI8fNCREJdIGHgRdGbg/edit?usp=sharing>

knowledge through experts, or fine-tuned LLMs tailored for the domain. Without clear software requirements, it would be challenging to produce accurate software design.

(4) *Evolvability of software requirements.* The evolution of requirements is the origin of software evolution. Prior studies [17, 174] more focus on fixed requirements, ignoring the evolvability of software requirements. How to incorporate LLMs into the adaptive perception and generation of evolving requirements has not received attention.

(5) *Comprehensive evaluation of generated requirements and design.* As defined in ISO 29148 [69], the quality of software requirements should be measured in various characteristics, including completeness, correctness, verifiability, unambiguousness, singularity, and feasibility, and so on [102]. The generated software design is expected to include concrete design specifications, **domain-specific languages (DSLs)**, and explore alternative architectures [166]. How to automate the evaluation of LLM-based software requirements and design process has received scant attention.

(6) *Inconsistency between software modeling and natural language descriptions.* Prior studies [10, 21, 24, 124] have demonstrated the software modeling capabilities of LLMs, such as generating **unified modeling languages (UMLs)** based on user stories, classifying model repositories, and so on. Although LLMs can generate syntactically correct UML code, they are struggling with semantic quality and tend to align poorly with natural languages, especially when the diagram scale is large.

*Summary.* The area of LLM for requirement and design still faces several challenges, including effective requirement/design prompts, structured descriptions of the communication between domain experts and software developers, lack of domain knowledge, dynamics of software requirements, comprehensive evaluation, and inconsistency in software modeling.

## 4.2 Challenges in Coding Assistance

During the software development cycle, the advent of code generation technologies powered by LLMs has greatly improved development efficiency and code quality, allowing developers to address business challenges instead of engaging in monotonous coding tasks. Despite the opportunities these advanced models introduce to code generation, they also present notable challenges.

(1) *Inaccurate generated code caused by hallucination in LLMs.* Even the most advanced LLMs may generate results that contradict the facts [23], compromising their reliability in code generation applications. This problem becomes especially critical in high-risk settings where strict compliance with specific application requirements is essential. The potential for inaccuracies in these environments underscores the substantial risks of deploying such models in production scenarios.

(2) *Introduction of vulnerabilities in generated code.* The pre-training datasets for LLMs are primarily from open source code repositories. Contributors to these repositories come from different backgrounds and possess varying levels of expertise, leading to numerous security vulnerabilities in the code. When LLMs learn from such datasets, they may unintentionally replicate these security issues in their outputs [153]. Deploying models trained on these data in real-world development projects can inadvertently introduce these vulnerabilities, posing risks with unforeseen consequences.

(3) *Limited generation performance for new programming languages.* The advent of emerging programming languages such as Rust presents a challenge due to the scarcity of available training data. Despite an urgent need for programming assistants to lower the barrier to entry and promote these new languages, the contradiction between the scarcity of training data and the pressing need for promotion poses a considerable challenge in creating effective LLMs-based code generation assistants.



(4) *Insufficient evaluation methods for machine-generated code.* Currently, the evaluation of code generation models primarily utilizes datasets like HumanEval [25] and MBPP [11], characterized by their simple and direct test cases. However, real development scenarios demand that models handle complex real-world code involving interactions across multiple methods and files to complete tasks [34]. To effectively evaluate the capabilities of LLMs in code generation, it is crucial to incorporate this complexity of real-world development environments.

(5) *Efficient integration of the generated code into projects.* Currently, most of the code generated by an LLM can solve common coding problems, e.g., algorithm problems. However, when given a coding task (e.g., adding a new feature or fixing a bug) in a real project, developers need to first identify the locations where they need to write/modify the code, and then complete the project-specific code [183]. Note that the newly added code should follow the coding style of the current project and call relevant APIs within the project. It is challenging for LLMs to well capture the project-level knowledge, remember the relevant APIs, identify the coding positions, and provide maintainable code with same coding styles.

*Summary.* For code generation, it needs to overcome numerous challenges posed by LLMs, such as issues with accuracy of generation, security vulnerabilities, insufficient support for new programming languages, inadequacies in evaluation methods, and efficient integration of the generated code into projects, to ensure the effective and safe application of LLMs.

### 4.3 Challenges in Testing Code Generation

The remarkable performance capability of LLMs offers new opportunities for research in software testing, especially unit test generation. The industry currently utilizes LLMs for unit test generation mainly involving two steps: first combining the method under test with its code context into a prompt for querying LLMs, and then refining the LLMs' output through text post-processing to ensure the successful compilation and execution.

The challenges for high-quality test cases are twofold, encompassing syntactics and semantics aspects. Specifically, at the syntactics level, the test code must be compiled and executed correctly. At the semantic level, the test code should comprehensively cover the tested code and support defect detection throughout the process in SDLC for maintaining code quality. Although LLMs can generate test code currently [110, 180], providing high-quality test cases that satisfy both syntactical correctness and comprehensive semantic coverage remains a considerable challenge.

(1) *The syntactics-related challenge.* In unit testing, the generated test code often contains code elements that cannot be successfully parsed by compilers [132] (e.g., undeclared variable types or non-existent identifiers) and non-code texts (e.g., explanatory natural language text). This limits their practical application in software development, as developers cannot directly utilize these outputs. Therefore, the challenge is to guide or constrain LLMs to reduce texts that cause compilation errors and to enhance the syntax compilation success rate of generated test cases. Two solutions to address this challenge include model fine-tuning (**supervised fine-tuning (SFT)**) [130], which manually collects and annotates test code with specific formats to fine-tune the model for satisfying syntax requirements, and prompt engineering, which involves instructions in the prompts to guide the model not to output irrelevant code/texts. However, developing these techniques to keep pace with the evolving capabilities of LLMs remains a significant challenge.

(2) *The semantics-related challenge.* First, when ensuring that the test code can be compiled successfully, we then need to inspect whether the test cases can cover all executable paths of the code under test. To achieve comprehensive path coverage, test cases are expected to be diverse. This involves specifying preconditions such as inputs, external dependencies, and environmental states

for the method being tested. For analyzing these preconditions, various testing techniques can be considered, such as boundary value testing, equivalence partitioning testing, and random testing. Accurately extracting or analyzing these preconditions poses a challenge, which could be addressed by using program analysis techniques to obtain a set of constraints for the preconditions. Another challenge is appropriately assembling these preconditions into prompts [169] to guide the model in outputting test cases with high coverage. Exploring various prompt engineering techniques could provide solutions for incorporating preconditions effectively.

Second, for large-scale projects with many dependencies, developers frequently mock the objects to simulate the behavior of the real objects. Thus, when applying SE-specific LLMs to generate test cases, it is challenging for the LLMs to identify the mocking objects automatically, and prefill the data into the mocked objects to simulate the practical behavior.

Third, high-quality code is characterized not only by correct implementation logic but also by returning correct results, appropriately modifying relevant variables, and capturing and handling exceptions, all in alignment with the intended code design. Unit test cases use assertion statements to verify the expected correctness of the code under test. However, providing LLMs with necessary information to generate test cases that include effective assertion statements poses a challenge. A promising approach is mining or deducing the design requirements of the code under test from historical test data to establish an accurate test oracle.

*Summary.* For unit test generation, it faces two primary challenges: syntactical correctness and comprehensive semantic coverage. Syntactically, the test code must be compilable and executable. Semantically, it should cover all execution paths and accurately verify the code's intended functionality.

#### 4.4 Challenges in Code Review

Code review refers to the process where developers, apart from the coders, meticulously examine the modifications made in a code commit request. In traditional workflows, these critical code review activities require significant human effort and call for review outcomes of exceptional quality [100]. The rapid development of LLMs has brought new opportunities for automating code review with some human intervention and guidance. However, due to the lack of knowledge base of code review, which heavily relies on specific requirements and objectives, complete automation and intelligence in code review are not yet achievable [145]. The challenges of LLMs in code review applications primarily manifest in four aspects: (1) the lack of evaluation and refinement of high-quality code review comments, (2) construction of code review applications tailored to different types of issues, (3) gap between code review practices in industry and open source communities, and (4) end-to-end automation of the code review process.

(1) *Constructing high-quality code review data by evaluation and optimization.* Compared to tasks like code generation, existing LLMs lack optimization. It is necessary to use high-quality code review-specific data to fine-tune LLMs, enabling them to better adapt to code review tasks. However, the quality of code review feedback data varies in real-world projects. For example, the feedback heavily relies on other comments (e.g., “same as above” and “refer to the previous comment”), leading to unclear objectives and expressions [78]. Therefore, it is crucial to design an evaluation method to select high-quality code review data and propose an optimization approach to refine low-quality code review comments.

(2) *Building code review applications for different types of issues.* In real-world scenarios, code changes submitted for review often involve various issues, including security vulnerabilities, coding standard violations, and general lower-priority code formatting issues [175]. Existing code review

applications based on LLMs handle all problem types as a generic code review, which causes LLMs to primarily focus on finding more common problems, such as standard and formatting issues, making it difficult to detect more severe problems. Furthermore, due to the nature of SFT and the corresponding relation between a set of code changes and a single code review feedback [89], building code review applications for different types of problems is a significant challenge in the code review process.

(3) *Mitigating the gap between code review practices in industry and open source communities.* Existing code review research related to LLMs primarily focuses on open source software projects. However, there are differences between code review practices in open source communities and industry. Specifically, developers in the industry are most concerned about security issues that directly impact the developers and the product's viability [15], and open source community projects involve fewer security issues and more diverse formatting and documentation-related problems [140]. Therefore, addressing key industry requirements, acquiring LLMs post-training based on internal data, and effectively utilizing prompt engineering for inference are crucial challenges for industry-based code review applications.

(4) *End-to-end automation of the code review process.* Current code review automation technologies mainly focus on automating specific activities within the code review process. These existing technologies are primarily based on deep learning and pre-training models [89, 144], requiring fine-tuning on specific datasets for different downstream tasks and then finishing the whole code review process by stages. In the era of LLMs, achieving full automation of the code review process has become an exploratory challenge. For example, when a developer proposes a requirement, the LLMs can generate problem tickets from the requirement, locate the corresponding code, automatically generate fix solutions, and ultimately achieve end-to-end automation and intelligence from requirement proposal to problem resolution.

*Summary.* The challenges in code review applications can be delineated across four aspects: (1) There is a lack of methods to evaluating and refining real-world code review comments for constructing high-quality code review datasets; (2) the development of specialized code review tools is constructed to address different types of issues; (3) the gap in the purpose and process exists between industrial practices and open source communities; (4) it lacks an end-to-end automation pipeline within the code review process.

## 4.5 Challenges in Software Maintenance

In modern software maintenance, adopting LLMs can enhance modular management and data-driven decision-making capabilities of existing approaches, thereby improving the maintenance efficiency. However, despite their potential to handle complex systems, they still face many challenges in practical operational scenarios: (1) the complexity of service dependencies in microservice architectures, (2) the lack of high-quality private operational data, and (3) the need for interpretability and high reliability in operational information.

(1) *Complexity of service dependencies in microservice architectures.* In microservice architectures, systems are divided into multiple independently operating service units. This design strategy greatly enhances system modularity and scalability. However, it also leads to fragmented contextual information and increases the complexity of dependencies between service units [133]. The dispersion of information poses challenges to LLMs because they rely on continuous and consistent contextual information for accurate decision-making and prediction. The dependencies between microservices are complex and dynamic, further limiting a single model's ability to capture the full scope of cross-service operations or transactions. Therefore, to achieve effective operational management in

microservice architectures, there is an urgent need to develop innovative technologies to enhance LLMs' ability to integrate dispersed contextual information and improve their understanding of complex inter-service dependencies.

(2) *Lack of high-quality private operational data.* In managing large-scale systems, performance optimization is crucial, especially when handling massive data and high user volumes. However, the insufficient quality and quantity of operational data make LLMs difficult to effectively optimize performance and accurately predict potential system bottlenecks. This issue is particularly prominent in industrial environments which involve highly proprietary data and specific system requirements [119]. These private data, due to sensitivity and confidentiality, are usually not publicly available, further limiting training data sources for LLMs. This confidentiality also leads to uneven data quality, deepening the difficulty for LLMs to perform real-time monitoring and optimization in industrial scenarios.

(3) *Insufficient interpretability and reliability of the results.* For ensuring the software reliability [43], LLMs-based approaches face unique challenges. First, LLMs must avoid generating inaccurate or false information when handling complex operations tasks, as this can lead to serious consequences. Also, since operational decisions directly impact system performance, all results must be easy for operation teams to understand and verify [139]. The opaque nature of LLMs may limit their use in high-reliability environments.

*Summary.* The area of LLMs for software maintenance still faces several challenges, including the complexity of service dependencies in microservice architectures, lack of high-quality private operational data, and need for interpretability and reliability of the operational information.

#### 4.6 Challenges in Vulnerability Management

The rapid development of LLMs is bringing new possibilities for building and enhancing vulnerability management capabilities. The major scenarios of vulnerability management applications include perception, validation, explanation, analysis, and remediation for vulnerability scenarios [80]. However, LLM-based vulnerability management applications are still in the preliminary validation and exploration stage [47]. There are several important challenges to overcome before achieving large-scale, efficient, and reliable applications, including (1) the lack of understanding of vulnerability information within the LLMs; (2) the scarcity of high-quality vulnerability analysis data; (3) the complexity of vulnerability context information, which need to extract adequate slices as input and consider the window size of LLMs.

(1) *Lack of understanding of vulnerability information.* LLMs are typically trained on open source data from various Internet sources [5]. However, the availability and quantity of open source data containing vulnerability information are limited [51], resulting in an insufficient understanding. It impacts the accuracy and reliability of vulnerability management applications. Therefore, native LLMs require extensive vulnerability data to effectively develop specialized vulnerability management, highlighting the importance of data-driven training in the construction of corresponding applications.

(2) *Scarcity of high-quality vulnerability analysis and explanation data.* The strength of generative models lies in providing detailed analysis and explanations for problems. However, building vulnerability management applications based on LLMs lacks the high-quality vulnerability explanation and analysis data. It is due to that open source vulnerability information datasets mainly provide information such as code patches, CVE IDs, and CWE types [16], but it is less accessible for developers to understand and effectively address vulnerabilities. Therefore, obtaining high-quality

vulnerability analysis and explanation data for vulnerability management applications remains an important challenge.

(3) *Complexity of vulnerability context information.* It is also essential to provide sufficient vulnerability context information in the prompt [162]. This context includes the code location, vulnerability type, and detailed patch information. However, these pieces of information can be lengthy, potentially exceeding the window size of existing LLMs and leading to the loss of important information. LLMs may struggle to consider all aspects of vulnerabilities simultaneously, resulting in inaccurate assessments of severity and impact.

*Summary.* Vulnerability management applications have the following challenges: First, to better understand vulnerability information, LLMs need to be enhanced in their interpretability by training on extensive vulnerability data. Second, high-quality vulnerability analysis data are scarce. It is important to obtain high-quality vulnerability analysis and explanation data for vulnerability management applications. Finally, current LLMs may not fully capture the extensive and intricate context surrounding vulnerabilities, which need to extract adequate slices as input while considering the limited window size.

#### 4.7 Challenges in Data, Training, and Evaluation

In this section, we discuss the challenges related to data construction, model training, and performance evaluation, which could impact the integration of LLMs into the software development process in practice.

(1) *High-quality code data.* The LLMs have successfully been applied in code generation, which attracts considerable research attention toward developing LLMs specialized in SE for downstream tasks. Despite the rapid evolution and iteration of LLMs, their core architecture is primarily based on the Transformer model. In this scenario, high-value data sources and high-quality pre-training data have become key factors for enhancing LLM performance [77] and could become companies' core commercial assets. Therefore, constructing high-quality code data presents numerous challenges.

During the pre-training phase of code LLMs, the scale and diversity of data are two crucial factors affecting model performance. Open source code communities (such as GitHub and Stack Overflow) have become the primary data sources. However, the quality of data from these communities varies greatly. Publicly available datasets in SE often lack rigorous quality assurance. Additionally, the current practices for source code cleaning typically rely on heuristic text cleaning rules [184], which mainly address format issues but fall short of ensuring data consistency, diversity, accuracy, and completeness. These practices also do not effectively utilize the structural and semantic features of code. *Therefore, the challenge of creating a large-scale, diverse, and high-quality code dataset during the pre-training phase and defining standards for high-quality code data remains unresolved.*

During the fine-tuning phase for various downstream tasks, high-quality and labeled data are essential for enhancing the performance of code LLMs. Key challenges include identifying relevant code snippets and their contexts within extensive code datasets for effective fine-tuning, as well as accurately labeling data while minimizing manual labor costs. Furthermore, it is critical to refine existing datasets for downstream tasks to ensure their quality and expand these datasets to encompass a broader range of SE tasks. Although advanced models, such as the GPT family, offer support for data annotation [32], their effectiveness in generating consistent and robust code remains challenging due to several factors. Addressing these issues is critical for improving the utility and accuracy of code LLMs in solving different SE tasks.

(2) *Exhaustive training.* Training LLMs present challenges such as high costs and poor stability. The lack of advanced engineering capabilities for training further exacerbates these issues. Companies



like Meta and OpenAI have expressed concerns about the excessive proportion of computational resources dedicated to training debugging, which further drives up the cost of training large models. Existing model training dashboards, such as TensorBoard [2], only provide basic training features and offer limited assistance to model training personnel in debugging models. Therefore, the training process requires advancements in deployment, configuration, bug localization, and process monitoring.

(3) *Comprehensive evaluation*. Evaluating code LLMs involves two critical phases: the *model-selection phase* and the *post-selection phase*, where they focus on optimizing research and development efficiency in practical applications. Each phase introduces different challenges.

During the model-selection phase, evaluation typically relies on benchmark datasets. However, existing studies indicate that the performance of LLMs on established benchmarks (e.g., HumanEval and MBPP) has reached near-limit bottlenecks as these models evolve, making it difficult to conduct comprehensive evaluations. Recently, evaluation datasets derived from real-world development projects have been released [44], yet their creation demands considerable manual annotation and verification effort. Challenges in this phase include setting evaluation standards, efficiently constructing high-quality benchmark datasets, and ensuring these datasets comprehensively represent the scope of evaluation.

The evaluation of code LLMs in real-world usage presents distinct challenges. Benchmark-based scores (such as hit rate, coverage rate, and edit distance) do not accurately capture the models' practical utility to developers. Developing evaluation frameworks that mimic real development contexts and feeding the evaluation of generated content into the model training phase to identify training or data issues represent advanced requirements and remarkable challenges in model evaluation.

*Summary.* For the training data, it needs to create a large-scale, diverse, and high-quality code dataset to boost code LLMs' effectiveness. Training LLMs also presents high costs and poor stability due to the lack of advanced engineering capabilities for training debugging. Regarding the evaluation part, it is essential to establish frameworks that replicate real-world development settings, allowing for a thorough and precise measurement of the models' practicality.

## 5 Related Work

### 5.1 LLM-Based Software Design

In recent works, LLMs have been used in various subtasks of requirement engineering, including anaphoric ambiguity treatment, requirement classification, requirement terminology recognition, coreference detection, and traceability automation.

For anaphoric ambiguity treatment, Ezzni's empirical study [40] verifies that SpanBERT excels at coreference explanation but has limited performance in ambiguity detection. Moharil and Sharma [108] introduce a toolkit called TABASCO to detect and recognize intra- or inter-domain ambiguity. It utilizes BERT for representation computation and clustering algorithms for analysis, proving the effectiveness of BERT in ambiguity detection. Sridhara et al. [134] apply ChatGPT to multiple SE tasks. In requirement engineering, ChatGPT successfully identifies all antecedents in the requirements for coreference detection, preliminarily verifying the applicability of ChatGPT to enhance the clarity of requirement expressions and eliminate ambiguity to some extent.

During project initialization, requirements often need to be categorized, such as distinguishing security-related requirements from others. They may also be divided based on relevance to functionality. Moreover, specific terminologies and entities in requirements may have different meanings in different contexts. NorBERT [58] is a fine-tuned BERT that takes advantage of BERT's strength in



transfer learning and achieves huge improvements in identifying functional and non-functional requirements compared to previous approaches. PRCBERT [103] benefits from prompt tuning, which enhances BERT's classification ability, outperforms NoRBERT, and demonstrates notable capabilities in zero-shot inference on other datasets. Moharil et al. [107] extract terminologies and synonyms from requirements with BERT and clustering algorithms, showcasing sample sentences from the corpus to clarify the meanings of terminologies within specific contexts. DeepCoref [155] employs fine-tuned BERT and a Word2Vec model to represent entities and predict associations between two entities.

Furthermore, LLMs are widely adopted to solve other requirement engineering tasks, like traceability automation, which establishes and maintains relationships among requirements, code, test samples, and other elements in the software system. T-BERT [92] is a framework that can create tracing links between code and natural language descriptions. Due to limited training data, traditional deep learning models often underperform pre-trained models in this task. Poudel et al. [122] release Sat-BERT and DSat-BERT for requirement satisfaction assessment, providing more accurate results than information retrieval approaches. Ronanki et al. [125] use ChatGPT to evaluate the quality of user stories, which shows that the judgments from ChatGPT align well with human evaluations. SpecGen [105] leverages LLMs to verify the formal specifications of programs. It first generates specifications for a given program and employs multiple mutations to refine the generated specifications. Xie et al. [168] conduct an empirical study on utilizing LLMs to generate software specifications with few-shot learning, which evaluates the performance and costs of 15 LLM-generated specifications and emphasizes the importance of prompt design and domain knowledge. Endres et al. [39] convert informal natural language descriptions into assertions with LLMs, and verify the correctness of generated assertions, which can help identify defective code. Zhang et al. [189] propose a new development practice, AISD, which first accepts user requests and then generates detailed applicable scenarios and prototype system designs before proceeding to generate the system implementation.

## 5.2 LLM-Based Code Assistance and Generation

Code generation can be applied to various scenarios, including code completion, code search, bug fixing, code review, and automatic testing. These applications aim to enhance development efficiency, reduce human errors, and assist developers in implementing new features rapidly. In recent years, code LLMs have been widely applied to code generation tasks. These models adopt different architectures, including CodeBERT [45], PLBART [4], and CodeGPT [101]. They are pre-trained on code corpora to gain a deep understanding of code syntax, semantics, and common structures. To boost their understanding of code complexity, some innovative methods integrate structural representation. GraphCodeBERT [53] incorporates graph-based representation on top of CodeBERT, while CodeT5 [156] combines the Encoder-Decoder paradigm with code structure. These enhancements aim to provide more fine-grained understandings of code relations and dependencies for the model in addition to syntax information.

Open source communities have provided abundant resources for code generation research, such as open source repositories, datasets, and tools. For instance, GitHub, Stack Overflow, and other platforms offer plenty of code samples, which are used to train and evaluate LLMs. CodeX [1] and CodeGen [114] are constructed with billions of parameters, demonstrating state-of-the-art performance on code generation tasks, capable of assisting the developers to generate code and improve programming efficiency. The success of CodeX prompts the development of similar models like StarCoder [86] and CodeLLaMA [127]. Developers can also use ChatGPT and GPT-4 to generate code, which has been proven effective in such fields.

However, instead of developing new LLMs, existing studies usually exploit LLMs as black-box tools. They integrate many techniques of **in-context learning (ICL)**, multi-agent systems, integrating traditional methods, user interaction, and CoT reasoning to improve the effectiveness of LLMs in solving different tasks.

Jiang et al. [72] propose a two-stage code generation pipeline based on ICL, i.e., *planning* and *implementation*. This pipeline allows LLMs to plan the solution steps according to users' intent before generating code. Li et al. [84] propose a new framework, AceCoder, which retrieves similar code as cases so that LLMs can learn algorithms and API knowledge. It then instructs LLMs to generate test cases and APIs to ensure that they understand the requirements before generating the code. LAIL [83] trains an auxiliary retriever to fetch examples for ICL, which first estimates the probability of generating the sample programs given the requirements and inputs with LLMs, then labels samples as positive or negative. It initiates contrastive learning for the retriever to learn LLM preferences, so it can select the optimal examples for ICL. On the other hand, as inserting print statements is effective for debugging, Hu et al. [61] utilize this debugging approach and instruct LLMs to insert print statements with ICL, execute the code locally, and ask LLMs to refine the code based on the outputs.

Multi-agent systems play different roles in accomplishing tasks collaboratively. These roles often mimic real-world development processes. Dong et al. [36] point out that code generation research should not be confined to programming alone, and suggest adopting multiple LLM agents to simulate different development team roles, such as analysts who break down user requirements, developers who generate or update code, and testers who test the code from various aspects. AgentCoder [64] adopts three agents, including a developer, a test designer, and a tester. The developer agent generates and refines the code based on tester feedback, the test designer agent generates test cases given the code, and the tester agent executes the test and offers feedback. LCG [91] simulates real-world development processes with multiple agents, including requirement engineers, architects, developers, and testers, and improves the development performance of GPT-3.5 with CoT and prompt engineering techniques. Similarly, MapCoder [68] employs four agents, each responsible for collecting relevant cases, planning, generating code, and debugging, respectively.

As LLM struggles with repository-level code generation, some researchers combine traditional tools to bridge the gap. CodePlan [13] studies repository-level code generation, constructing planning graphs based on dependency analysis and adaptive planning, with edit positions as nodes and edit orders as edges. Information from these graphs is integrated into the prompts used to generate code. The code is then merged into the repository, causing an update to the planning graphs as the process continues iteratively. ToolGen [151] integrates auto-completion tools to resolve the dependency issues in repository-level code generation. It first enhances the code in the repository with special tokens for LLM fine-tuning alongside documentation. During token sequence generation, the auto-completion tool produces the correct completion entry when the LLM generates a particular token. Zhang et al. [187] release ToolCoder, which fine-tunes LLMs using a ChatGPT-augmented dataset with tool-use information. During inference, an API search tool is integrated to offer suggestions on API selection. Zhang et al. [186] propose CodeAgent, integrating five tools for tasks like information retrieval and code symbol navigation, as well as four agent strategies like OpenAI's function calling, to help LLMs generate code. However, not all studies report state-of-the-art performance for LLMs. For example, Bochenek [18] tries to use ChatGPT to complete Java code templates. In conclusion, the author claims that the results are neither predictable nor reproducible, and traditional template-filling approaches perform better.

Some studies investigate the interactions between users and LLMs. Fakhoury et al. [41] propose a novel framework, TiCoder, to ensure the correctness of generated code. It first generates code and test cases with LLMs, tests the code locally, asks LLMs to select the best test cases based on the test results, and finally requests the users to evaluate the correctness of the code, test cases, and test results. Unsatisfactory test cases are then pruned, and the framework updates the code and test cases iteratively. Considering the lack of domain knowledge, Yan et al. [172] propose IntelliExplain, which requires LLMs to explain the users' intent in detail in natural language and avoid code generation until the user agrees with the LLMs' explanation.

To mitigate misalignment between code and specifications in previous CoT methods, Tian et al. [142] design a test case-driven CoT approach named TCot, which lets LLMs understand specifications through test cases before generating the code. Sun et al. [137] propose CodePlan, which uses multi-task learning to distill the ability of code generation and planning from larger LLMs to smaller ones.

Other studies examine code generation from perspectives beyond the correctness of the code. Nguyen et al. [111] construct GPTSniffer with CodeBERT, which helps to identify whether the code is generated by AI tools like ChatGPT. Huang et al. [63] investigate whether LLM-generated code contains biases related to age, race, and gender. It uses GPT-4 to detect biases and utilizes few-shot learning to reduce them. Niu evaluates the efficiency of generated code on existing datasets and explores how prompt strategies affect code efficiency.

### 5.3 LLM-Based Software Testing

In software testing, LLMs are typically used to generate test cases for automatic testing, construct assertions, produce test inputs, analyze defects, and help debug and fix the code. For example, LLMs can generate expected test cases by understanding code contexts and structures [171], to improve test coverage and effectiveness. Furthermore, LLMs can generate accurate assertions [104, 178] as part of test cases, to validate whether the behavior of the software meets the expectations. On the level of system testing, LLMs can generate various test inputs, which is essential for testing user interfaces and functionalities of mobile applications.

Recently, various approaches aim at generating unit tests with LLMs, which is often associated with pre-training or fine-tuning. Alagarsamy et al. [8] propose A3Test, a framework inspired by domain adaptation, which transfers assertion generation knowledge into test code generation. They pre-train LLMs with methods to be tested and assertion statements, equipping LLMs with more knowledge about assertions. The pre-trained models are then fine-tuned for test code generation, allowing them to better understand the relationship between the methods to be tested and their corresponding test code. Similarly, Hashtroudi et al. [57] pre-train LLMs with existing developer-written test code, helping models adapt to the new project domains and generate human-readable unit tests. Rao et al. [123] introduce CAT-LM, which uses a novel pre-train signal to map code to test files and train a 270M GPT-style language model. Steenhoek et al. [135] release RLSQM, which scores programs based on static analysis and embeds the quality information into LLMs, including factors such as the presence of assertion statements, whether the methods to be tested are invoked, and the existence of method signatures.

With the development of LLMs, they can exhibit satisfying performance without further pre-training or fine-tuning. Therefore, many recent researches instead focus on prompt strategies to improve LLMs' ability to handle context and details. Xie et al. [169] propose ChatUniTest, adopting a generate-verify-repair framework, which analyzes project information and extracts key data to create a context embedded with methods to be tested as well as their dependencies. After the LLMs receive the prompt and generate feedback, ChatUniTest extracts test code from the outputs, verifies the code, and fixes compilation errors with either rule-based or LLM-based approaches. Other

works generate unit tests with additional documentation. Plein et al. [120] suggest generating test code that can reflect real-world, complex usage scenarios, which exposes faulty program behavior. As such scenarios are often described informally in defect reports, they consider such reports as valid inputs to generate fault-triggering test code, unlike previous works focusing on generating random test inputs.

Researchers also develop many novel methods to generate test inputs or test oracles with LLMs. These inputs validate mobile apps, Web applications, and deep learning frameworks. To automate the process of mobile app testing, researchers propose different test input generation approaches with LLMs. Yoon et al. [177] propose DROIDAGENT, which sets goals with LLMs given the Android app and achieves the goals by interacting with the apps. The evaluation of DROIDAGENT shows that it can engage in deeper interaction with apps to cover more features. Liu et al. [96] propose QTypist, which uses LLMs to generate input text for mobile app GUIs, solving the challenge of generating varied and semantically correct GUI inputs. The evaluation shows that QTypist successfully improves the test pass rate and covers more app activities and pages. Liu et al. [97] proposed InputBlaster to generate abnormal text inputs for mobile apps with LLMs. It creates multiple test generators, each capable of generating abnormal inputs based on LLM-generated mutation rules.

In Web test automation, Kim et al. [76] propose RESTGPT, generating test inputs for REST API with LLMs, which overcomes the limitations of previous methods. It struggles to extract rule types from unstructured natural language and improves accuracy. Another issue with LLM-generated programs is the failure to cover corner cases. To solve this issue, Deng et al. [31] release FuzzGPT, which pre-processes LLMs and synthesizes fault-triggering test inputs for fuzzing. FuzzGPT automatically extracts historical knowledge of fault-triggering programs with LLM capabilities. Evaluation results prove FuzzGPT's effectiveness in fuzzing and detecting more errors.

In other domains, researchers also propose innovative methods for LLM-based test input generation. For example, Baudry et al. [14] highlight the ability of LLMs to generate pseudo-test data and successfully generate data aligning well with cultural contexts and suitable for testing, as well as code compatible with test input generation tools. Xia et al. [167] propose Fuzz4All, a generic fuzzing tool using LLMs as an engine to generate and mutate test inputs. Fuzz4All supports multiple programming languages and scenarios, such as C/C++ compilers, solvers, tools in Go, Java compilers, and quantum computation platforms.

Test oracles can validate the correctness of test results. In traditional approaches, test oracles are usually derived from specifications. However, manual generation is labor-intensive, while automatic generation proves to be extremely challenging. In recent years, researchers have started to use LLMs to help generate test oracles. **Metamorphic testing (MT)** has emerged as a successful solution to automated testing and test oracle generation. MT relies on **metamorphic relations (MRs)**, which describe the relationship between test inputs and outputs. Shin et al. [131] utilize LLMs to infer **executable MRs (EMRs)** from requirements, guiding them to understand software specifications and DSLs for EMRs to generate the test oracle. Zhang et al. [190] explore applying ChatGPT to generate MRs automatically and evaluate their quality for auto-driving systems, demonstrating the effectiveness of the proposed approach, which significantly reduces manual effort in MR generation. Tsigkanos et al. [143] design an LLM-based process to extract variables from user manuals of scientific software, achieving an accuracy of 0.87 and successfully extracting 61.8% partially matching variables as well as 34.7% exact-matching variables. Hyun et al. [67] introduce METAL, promoting systematic evaluation of LLM quality with MT. METAL can automatically generate hundreds of MRs from templates covering various qualitative properties and tasks. Furthermore, they introduce new metrics, which combine attack success rate and semantic quality, to better evaluate the effect of MT.

#### 5.4 LLM-Based Code Review

The applications of LLMs in code analysis and review are a trending topic in the current field of artificial intelligence. These models utilize deep learning techniques to learn a large amount of code data so that they can understand and generate programs with complicated structures. These LLMs play an important role in software development.

In code analysis, researchers have developed multiple models to understand code diffs and reviews. Universal models like GPT-3 display outstanding performance in several natural language processing tasks, but on the other hand, might be less accurate in code analysis compared to specialized models. Specialized models, such as CodeBERT and GraphCodeBERT, are specifically built on understanding code semantics, pre-trained on large code corpora, and can better capture code syntax and semantics.

In code review, LLMs are mainly applied to review code and detect flaws automatically. For instance, DeepCode [75] is a code review tool powered by machine learning, which can detect security vulnerabilities and performance issues to help developers enhance the quality of code. CodeReviewer [89] is a pre-trained model tailored specifically for the code review scenario, which is trained on a large-scale dataset of real-world code changes and code reviews from open source projects, helping to better understand code diffs and reviews. Tufano et al. [146, 147] approach code review from the perspectives of both the developer and the reviewer, and train T5 for two subtasks of code review. The first subtask requires the model to refine the given code, while the second requires the model to edit the code based on a review comment. AUGER [85] is a framework utilizing the pre-trained model CodeTrans, a variant of T5. It labels lines of code relevant to the review comment, performs cross-pre-training between functions and comments, and fine-tunes for comment generation. AutoTransform [141] addresses new tokens with Byte Pair Encoding and adopts Transformer in a machine translation manner to edit the code after review. Zhou et al. [193] study previous generative code review methods and conclude that: (1) They are based on different datasets, making comparisons difficult; (2) There are few studies targeting CodeT5; (3) Exact match is often the primary metric, while other metrics are overlooked. To address these issues, they adopt CodeT5, introduce a new metric called edit process, and compare their approach with previous works.

LLaMA-Reviewer [100] uses LLaMA in code review, utilizing parameter-efficient fine-tuning to reduce resource consumption while maintaining high performance. As a result, even the smallest LLaMA-6.7B performs comparably to previous methods. Wen et al. [160] use GPT-3.5 for code analysis and review. They first extract relevant code with static tools, bug reports, and program dependencies, before constructing formal prompts that incorporate domain knowledge and representative examples, and prompting the LLM to evaluate the accuracy of warnings from static tools, greatly reducing false positives. Pornprasit et al. [121] also use GPT-3.5 for code review, investigating the performance impact of few-shot learning, role specification, and model fine-tuning, and compare the results with previous approaches. Tufano et al. [145] study whether current methods are sufficient for comment generation and comment-based code editing, examining the types of samples where these methods succeed or fail and comparing them with ChatGPT.

#### 5.5 LLM-Based Software Maintenance

As software systems grow larger, the maintenance becomes more complex and critical. The introduction of LLMs brings new opportunities to software maintenance, such as failure identification [66, 73, 81, 106], root cause analysis [6, 26, 74, 157, 182], and postmortem analysis [35, 48, 65].

For failure identification, researchers use LLMs for log parsing and anomaly detection, addressing challenges in semantic understanding and efficiency. For example, Huo et al. [66] first consider



semantic information in log parsing. Le et al. [81] try to use ChatGPT for log parsing and initially confirm the ability of LLMs in this area. However, LLMs have issues such as long response times and high computational costs when processing a large number of logs. To address this, Jiang et al. [73] propose an adaptive caching mechanism that caches log templates to avoid repeated requests and improve parsing efficiency. Ma et al. [106] combine external knowledge to enhance log analysis. For root cause analysis, LLMs help understand system events and diagnose faults by combining code and log data for automatic analysis and recommendations. Ahmed et al. [6] use LLMs to automatically output fault root causes and suggest mitigation measures. Wang et al. [157] propose a tool-learning method. Zhang et al. [182] introduce a confidence evaluation framework to alleviate issues like model hallucination. Jin et al. [74] use LLMs for fault impact assessment and summary generation. Chen et al. [26] develop RCACopilot to achieve an end-to-end fault handling process. For postmortem analysis, LLMs assist in summarizing incident tickets, improving data labeling efficiency, and promoting fault prediction. Dogga et al. [35] propose fault classification rules and attempt automatic classification. Ganatra et al. [48] apply LLMs to the fault analysis pipeline. The Huawei Cloud team [65] proposes a hierarchical representation method to achieve automatic classification and prediction.

Overall, the application of LLMs in software maintenance improves efficiency, simplifies complex tasks, and reduces the time for repairing. However, it also brings challenges like complexity of service dependencies and model interpretability that need to be addressed.

## 5.6 LLM-Based Vulnerability Management

Recently, many studies have proposed to manage vulnerabilities, including vulnerability detection [163, 164, 165, 185], vulnerability assessment [116], and vulnerability awareness [117, 192].

Vulnerability detection aims to detect vulnerabilities in the source code [161, 165]. Automated code vulnerability detection by using LLMs has gained increasing attention in recent years [60, 99]. EPVD [185] proposes an execution path selection algorithm and adopts a pre-trained model to learn the path representations. SVulD [112] constructs contrastive paired instances and uses the pre-trained model to learn distinguishing semantic representations.

To improve early sensing of vulnerabilities, many studies are proposed to make aware of vulnerabilities from issue reports [117] or silent fixes [192]. Open source software usually adopts a vulnerability disclosure policy which causes the majority of vulnerabilities to be fixed silently. Identifying silent fixes and their corresponding explanations is essential to sense vulnerabilities. Zhou et al. [192] propose an approach named ColeFunda, consisting of a Contrastive Learner and FunDa, a novel approach for Function change Data augmentation. ColeFunda integrates contrastive learning and BERT models to improve its effectiveness in identifying different vulnerabilities. To sense vulnerabilities early, Pan et al. [117] augment the BERT model with a memory component, which stores the external vulnerability knowledge from CWEs.

## 6 Threats to Validity

One threat is the completeness of the challenges. The summarized challenges are based on the discussion among 24 participants, which might not cover all the current challenges of LLM4SE. In this article, to ensure the integrity of the challenges, the discussion was designed to encompass all the processes in SDLC and included six thematic sessions. Each session lasted for around 4 hours to ensure that the discussion was as thorough as possible.

Another threat is related to the representativeness of the participants. Our solution is increasing diversity, e.g., 17 academic researchers and 7 industry practitioners. We believe we have made this threat have minimal impact on the results of our investigation.



## 7 Conclusions

LLMs are bringing significant changes to the SE field and are applied to various SE tasks, such as code generation, code review, and vulnerability management. However, a comprehensive understanding of the challenges of LLMs on SE is still in the early stages. This article highlights the challenges and opportunities in LLM4SE. We present seven challenges summarized in the “9th CCF Beautiful Lake Seminars.” These challenges include various aspects of the SDLC, e.g., requirements, coding, testing, and code review. This article also highlights challenges in building LLMs, including data preparation, training, and evaluation. We provide a research roadmap of LLM4SE and outline promising future research directions.

## References

- [1] The 9th CCF Beautiful Lake Seminars. 2024. Retrieved from <https://www.ccf.org.cn/xhhy/2024-03-15/816185.shtml>
- [2] TensorBoard. 2024. Retrieved from <https://www.tensorflow.org/tensorboard>
- [3] Jean-François Abramatic, Roberto Di Cosmo, and Stefano Zacchiroli. 2018. Building the universal archive of source code. *Communications of the ACM* 61, 10 (2018), 29–31.
- [4] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. In *2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT '21)*. Association for Computational Linguistics, 2655–2668.
- [5] Toufique Ahmed, Christian Bird, Premkumar T. Devanbu, and Saikat Chakraborty. 2024. Studying LLM performance on closed- and open-source data. arXiv:2402.15100. Retrieved from <https://arxiv.org/abs/2402.15100>
- [6] Toufique Ahmed, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann, Xuchao Zhang, and Saravan Rajmohan. 2023. Recommending root-cause and mitigation steps for cloud incidents using large language models. In *45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 1737–1749.
- [7] Muhammad Azeem Akbar, Arif Ali Khan, Najmul Islam, and Sajjad Mahmood. 2024. DevOps project management success factors: A decision-making framework. *Software: Practice and Experience* 54, 2 (2024), 257–280.
- [8] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2024. A3test: Assertion-augmented automated test case generation. *Information and Software Technology* 176 (2024), 107565.
- [9] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Muñoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, et al. 2023. SantaCoder: Don't reach for the stars! arXiv:2301.03988. Retrieved from <https://arxiv.org/abs/2301.03988>
- [10] Sathurshan Arulmohan, Marie-Jean Meurs, and Sébastien Mosser. 2023. Extracting domain models from textual requirements in the era of large language models. In *ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (MODELS '23 Companion)*. IEEE, 580–587.
- [11] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. arXiv:2108.07732. Retrieved from <https://arxiv.org/abs/2108.07732>
- [12] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen Technical Report. arXiv:2309.16609. Retrieved from <https://arxiv.org/abs/2309.16609>
- [13] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. 2024. CodePlan: Repository-level coding using LLMs and planning. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 675–698.
- [14] Benoit Baudry, Khashayar Etemadi, Sen Fang, Yogya Gamage, Yi Liu, Yuxin Liu, Martin Monperrus, Javier Ron, André Silva, and Deepika Tiwari. 2024. Generative AI to generate test data generators. arXiv:2401.17626. Retrieved from <https://arxiv.org/abs/2401.17626>
- [15] Tobias Baum, Olga Liskin, Kai Niklas, and Kurt Schneider. 2016. Factors influencing code review processes in industry. In *Fast Software Encryption Conference*. ACM, 85–96.
- [16] Guru Prasad Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software. In *17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '21)*. Shane McIntosh, Xin Xia, and Sousuke Amasaki (Eds.), ACM, 30–39.
- [17] Kelly Blincoe, Giuseppe Valetto, and Daniela E. Damian. 2013. Do all task dependencies require coordination? The role of task properties in identifying critical coordination needs in software projects. In *Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE '13)*. ACM, 213–223.
- [18] Adam Bochenek. 2023. Can ChatGPT replace a template-based code generator? In *FedCSIS (Communication Papers)*, 43–50.

- [19] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 1877–1901.
- [20] Saheed A. Busari and Emmanuel Letier. 2017. RADAR: A lightweight tool for requirements and architecture decision analysis. In *39th International Conference on Software Engineering (ICSE '17)*. IEEE/ACM, 552–562.
- [21] Javier Cámara, Javier Troya, Lola Burgueño, and Antonio Vallecillo. 2023. On the assessment of generative AI in modeling tasks: An experience report with ChatGPT and UML. *Software and Systems Modeling* 22, 3 (2023), 781–793.
- [22] Yihan Cao, Siyu Li, Yixin Liu, Zhiling Yan, Yutong Dai, Philip S. Yu, and Lichao Sun. 2023. A comprehensive survey of AI-generated content (AIGC): A history of generative AI from GAN to ChatGPT. arXiv:2303.04226. Retrieved from <https://arxiv.org/abs/2303.04226>
- [23] Mikael Chelli, Jules Descamps, Vincent Lavoué, Christophe Trojani, Michel Azar, Marcel Deckert, Jean-Luc Raynier, Gilles Clowez, Pascal Boileau, and Caroline Ruetsch-Chelli. 2024. Hallucination rates and reference accuracy of ChatGPT and Bard for systematic reviews: Comparative analysis. *Journal of Medical Internet Research* 26 (2024), e53164.
- [24] Kua Chen, Yujing Yang, Boqi Chen, José Antonio Hernández López, Gunter Mussbacher, and Dániel Varró. 2023. Automated domain modeling with large language models: A comparative study. In *26th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (MODELS '23)*. IEEE, 162–172.
- [25] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from <https://arxiv.org/abs/2107.03374>
- [26] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, et al. 2024. Automatic root cause analysis via large language models for cloud incidents. In *19th European Conference on Computer Systems (EuroSys '24)*. ACM, 674–688.
- [27] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24 (2023), 240:1–240:113.
- [28] Fenia Christopoulou, Gerasimos Lampouras, Milan Gritta, Guchun Zhang, Yinpeng Guo, Zhongqi Li, Qi Zhang, Meng Xiao, Bo Shen, Lin Li, et al. 2022. PanGu-Coder: Program synthesis with function-level language modeling. arXiv:2207.11280. Retrieved from <https://arxiv.org/abs/2207.11280>
- [29] Copilot. 2024. Retrieved from <https://copilot.microsoft.com/>
- [30] DeepSeek-AI, Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y. Wu, Yukun Li, Huazuo Gao, et al. 2024. DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence. arXiv:2406.11931. Retrieved from <https://arxiv.org/abs/2406.11931>
- [31] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2023. Large language models are edge-case fuzzers: Testing deep learning libraries via FuzzGPT. arXiv:2304.02014. Retrieved from <https://arxiv.org/abs/2304.02014>
- [32] Bosheng Ding, Chengwei Qin, Linlin Liu, Yew Ken Chia, Shafiq Joty, Boyang Li, and Lidong Bing. 2022. Is gpt-3 a good data annotator? arXiv:2212.10450. Retrieved from <https://arxiv.org/abs/2212.10450>
- [33] Yangruibo Ding, Jinjun Peng, Marcus J. Min, Gail E. Kaiser, Junfeng Yang, and Baishakhi Ray. 2024. SemCoder: Training code language models with comprehensive semantics. arXiv:2406.01006. Retrieved from <https://arxiv.org/abs/2406.01006>
- [34] Yangruibo Ding, Zijian Wang, Wasi Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, et al. 2024. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. In *Advances in Neural Information Processing Systems*.
- [35] Pradeep Dogga, Chetan Bansal, Richard Costleigh, Gopinath Jayagopal, Suman Nath, and Xuchao Zhang. 2023. AutoARTS: Taxonomy, insights and tools for root cause labelling of incidents in Microsoft Azure. In *2023 USENIX Annual Technical Conference (USENIX ATC '23)*. Julia Lawall and Dan Williams (Eds.), USENIX Association, 359–372.
- [36] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-collaboration code generation via ChatGPT. arXiv:2304.07590. Retrieved from <https://arxiv.org/abs/2304.07590>
- [37] GitLab Duo. 2024. Retrieved from <https://about.gitlab.com/gitlab-duo/>
- [38] Andrew Edwards-Jones. 2014. Qualitative data analysis with NVIVO. *Journal of Education for Teaching: International Research and Pedagogy* 40, 2 (2014), 193–195.
- [39] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. 2023. Formalizing natural language intent into program specifications via large language models. arXiv:2310.01831. Retrieved from <https://arxiv.org/abs/2310.01831>
- [40] Saad Ezzini, Sallam Abualhaija, Chetan Arora, and Mehrdad Sabetzadeh. 2022. Automated handling of anaphoric ambiguity in requirements: A multi-solution study. In *44th International Conference on Software Engineering*, 187–199.

- [41] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. LLM-based test-driven interactive code generation: User study and empirical evaluation. arXiv:2404.10100. Retrieved from <https://arxiv.org/abs/2404.10100>
- [42] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large language models for software engineering: Survey and open problems. In *IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE '23)*. IEEE, 31–53.
- [43] Joao Faustino, Daniel Adriano, Ricardo Amaro, Rubén Pereira, and Miguel Mira da Silva. 2022. DevOps benefits: A systematic literature review. *Software: Practice and Experience* 52, 9 (2022), 1905–1926.
- [44] Jia Feng, Jiachen Liu, Cuiyun Gao, Chun Yong Chong, Chaozheng Wang, Shan Gao, and Xin Xia. 2024. Complex-CodeEval: A benchmark for evaluating large code models on more complex code. arXiv:2409.10280. Retrieved from <https://arxiv.org/abs/2409.10280>
- [45] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics (EMNLP '20)*. Association for Computational Linguistics, 1536–1547.
- [46] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A generative model for code infilling and synthesis. In *the 11th International Conference on Learning Representations (ICLR '23)*. OpenReview.net.
- [47] Michael Fu, Chakkrit Kla Tantithamthavorn, Van Nguyen, and Trung Le. 2023. ChatGPT for vulnerability detection, classification, and repair: How far are we? In *30th Asia-Pacific Software Engineering Conference (APSEC '23)*. IEEE, 632–636.
- [48] Vaibhav Ganatra, Anjaly Parayil, Supriyo Ghosh, Yu Kang, Minghua Ma, Chetan Bansal, Suman Nath, and Jonathan Mace. 2023. Detection is better than cure: A cloud incidents perspective. In *31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, 1891–1902.
- [49] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, Hongyu Zhang, and Michael R. Lyu. 2023. What makes good in-context demonstrations for code intelligence tasks with LLMs? In *38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 761–773.
- [50] Gartner. 2023. Retrieved from <https://www.gartner.com/en/articles/gartner-top-10-strategic-technology-trends-for-2023>
- [51] Hamza Ghani, Jesus Luna, Abdelmajid Khelil, Najib Alkadri, and Neeraj Suri. 2013. Predictive vulnerability scoring in the context of insufficient information availability. In *2013 International Conference on Risks and Security of Internet and Systems (CRISIS)*. Bruno Crispo, Ravi S. Sandhu, Nora Cuppens-Boulahia, Mauro Conti, and Jean-Louis Lanet (Eds.), IEEE Computer Society, 1–8.
- [52] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. 2023. Textbooks are all you need. arXiv:2306.11644. Retrieved from <https://arxiv.org/abs/2306.11644>
- [53] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *9th International Conference on Learning Representations (ICLR '21)*. OpenReview.net.
- [54] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, et al. 2024. DeepSeek-Coder: When the large language model meets programming - The rise of code intelligence. arXiv:2401.14196. Retrieved from <https://arxiv.org/abs/2401.14196>
- [55] Yiyang Hao, Ge Li, Yongqiang Liu, Xiaowei Miao, He Zong, Siyuan Jiang, Yang Liu, and He Wei. 2022. AixBench: A code generation benchmark dataset. arXiv:2206.13179. Retrieved from <https://arxiv.org/abs/2206.13179>
- [56] Mark Harman. 2012. The role of artificial intelligence in software engineering. In *1st International Workshop on Realizing AI Synergies in Software Engineering (RAISE '12)*. IEEE, 1–6.
- [57] Sepehr Hashtroudi, Jiho Shin, Hadi Hemmati, and Song Wang. 2023. Automated test case generation using code models and domain adaptation. arXiv:2308.08033. Retrieved from <https://arxiv.org/abs/2308.08033>
- [58] Tobias Hey, Jan Keim, Anne Koziulek, and Walter F. Tichy. 2020. Norbert: Transfer learning for requirements classification. In *2020 IEEE 28th International Requirements Engineering Conference (RE)*. IEEE, 169–179.
- [59] John E. Hopcroft, Jeffrey D. Ullman, and Alfred Vaino Aho. 1983. *Data Structures and Algorithms*. Addison-Wesley, Boston, MA.
- [60] Sihao Hu, Tiansheng Huang, Fatih Ilhan, Selim Furkan Tekin, and Ling Liu. 2023. Large language model-powered smart contract vulnerability detection: New perspectives. In *5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA '23)*. IEEE, 297–306.
- [61] Xueyu Hu, Kun Kuang, Jiankai Sun, Hongxia Yang, and Fei Wu. 2024. Leveraging print debugging to improve code generation in large language models. arXiv:2401.05319. Retrieved from <https://arxiv.org/abs/2401.05319>

- [62] Xing Hu, Xin Xia, David Lo, Zhiyuan Wan, Qiuyuan Chen, and Thomas Zimmermann. 2022. Practitioners' expectations on automated code comment generation. In *44th IEEE/ACM 44th International Conference on Software Engineering (ICSE '22)*. ACM, 1693–1705.
- [63] Dong Huang, Qingwen Bu, Jie Zhang, Xiaofei Xie, Junjie Chen, and Heming Cui. 2023. Bias assessment and mitigation in LLM-based code generation. arXiv:2309.14345. Retrieved from <https://arxiv.org/abs/2309.14345>
- [64] Dong Huang, Qingwen Bu, Jie M. Zhang, Michael Luck, and Heming Cui. 2023. AgentCoder: Multi-agent-based code generation with iterative testing and optimisation. arXiv:2312.13010. Retrieved from <https://arxiv.org/abs/2312.13010>
- [65] Junjie Huang, Jinyang Liu, Zhuangbin Chen, Zhihan Jiang, Yichen Li, Jiazhen Gu, Cong Feng, Zengyin Yang, Yongqiang Yang, and Michael R. Lyu. 2024. FaultProfiT: Hierarchical fault profiling of incident tickets in large-scale cloud systems. In *46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '24)*. ACM, 392–404.
- [66] Yintong Huo, Cheryl Lee, Yuxin Su, Shiwen Shan, Jinyang Liu, and Michael R. Lyu. 2023. EvLog: Identifying anomalous logs over software evolution. In *34th IEEE International Symposium on Software Reliability Engineering (ISSRE '23)*. IEEE, 391–402.
- [67] Sangwon Hyun, Mingyu Guo, and M. Ali Babar. 2024. METAL: Metamorphic testing framework for analyzing large-language model qualities. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 117–128.
- [68] Md Ashrafur Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. MapCoder: Multi-agent code generation for competitive problem solving. arXiv:2405.11403. Retrieved from <https://arxiv.org/abs/2405.11403>
- [69] ISO/IEC/IEEE International Standard 15288-2023. 2023. Systems and software engineering—System life cycle processes, ISO/IEC/IEEE 15288:2023, 1–128. DOI: <https://doi.org/10.1109/IEEESTD.2023.10123367>
- [70] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. CURE: Code-aware neural machine translation for automatic program repair. In *43rd IEEE/ACM International Conference on Software Engineering (ICSE '21)*. IEEE, 1161–1173.
- [71] Siyuan Jiang, Jia Li, He Zong, Huanyu Liu, Hao Zhu, Shukai Hu, Erlu Li, Jiazhen Ding, Yu Han, Wei Ning, et al. 2024. aiXcoder-7B: A lightweight and effective large language model for code completion. arXiv:2410.13187. Retrieved from <https://arxiv.org/abs/2410.13187>
- [72] Xue Jiang, Yihong Dong, Lecheng Wang, Fang Zheng, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2023. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2023), 1–30.
- [73] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R. Lyu. 2023. LLMParse: A LLM-based log parsing framework. arXiv:2310.01796. Retrieved from <https://arxiv.org/abs/2310.01796>
- [74] Pengxiang Jin, Shenglin Zhang, Minghua Ma, Haozhe Li, Yu Kang, Liquan Li, Yudong Liu, Bo Qiao, Chaoyun Zhang, Pu Zhao, et al. 2023. Assess and summarize: Improve outage understanding with large language models. In *31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, 1657–1668.
- [75] Hyeji Kim, Yihan Jiang, Sreeram Kannan, Sewoong Oh, and Pramod Viswanath. 2018. Deepcode: Feedback codes via deep learning. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018 (NeurIPS '18)*, 9458–9468.
- [76] Myeongsoo Kim, Tyler Stennett, Dhruv Shah, Saurabh Sinha, and Alessandro Orso. 2024. Leveraging large language models to improve REST API testing. In *2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, 37–41.
- [77] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. 2022. The stack: 3 tb of permissively licensed source code. arXiv:2211.15533. Retrieved from <https://arxiv.org/abs/2211.15533>
- [78] Oleksii Kononenko, Olga Baysal, Latifa Guerrouj, Yaxin Cao, and Michael W. Godfrey. 2015. Investigating code review quality: Do people and participation matter? In *International Conference on Software Maintenance and Evolution (ICSME)*. IEEE Computer Society, 111–120.
- [79] Thomas D. LaToza, Gina Venolia, and Robert DeLine. 2006. Maintaining mental models: A study of developer work habits. In *28th International Conference on Software Engineering (ICSE '06)*. Leon J. Osterweil, H. Dieter Rombach, and Mary Lou Soffa (Eds.), ACM, 492–501.
- [80] Triet Huynh Minh Le, Huaming Chen, and Muhammad Ali Babar. 2023. A survey on data-driven software vulnerability assessment and prioritization. *ACM Computing Surveys* 55, 5 (2023), 100:1–100:39.
- [81] Van-Hoang Le and Hongyu Zhang. 2023. Log parsing with prompt-based few-shot learning. In *45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 2438–2449.
- [82] Yoav Levine, Noam Wies, Daniel Jannai, Dan Navon, Yedid Hoshen, and Amnon Shashua. 2021. The inductive bias of in-context learning: Rethinking pretraining example design. In *International Conference on Learning Representations*.

- [83] Jia Li, Ge Li, Chongyang Tao, Huangzhao Zhang, Fang Liu, and Zhi Jin. 2023. Large language model-aware in-context learning for code generation. arXiv:2310.09748. Retrieved from <https://arxiv.org/abs/2310.09748>
- [84] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2023. Towards enhancing in-context learning for code generation. arXiv:2303.17780. Retrieved from <https://arxiv.org/abs/2303.17780>
- [85] Lingwei Li, Li Yang, Huaxi Jiang, Jun Yan, Tiejian Luo, Zihan Hua, Geng Liang, and Chun Zuo. 2022. AUGER: Automatically generating review comments with pre-training models. In *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1009–1021.
- [86] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: May the source be with you! arXiv:2305.06161. Retrieved from <https://arxiv.org/abs/2305.06161>
- [87] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023. Textbooks are all you need II: Phi-1.5 technical report. arXiv:2309.05463. Retrieved from <https://arxiv.org/abs/2309.05463>
- [88] Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with AlphaCode. arXiv:2203.07814. Retrieved from <https://arxiv.org/abs/2203.07814>
- [89] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, et al. 2022. Automating code review activities by large-scale pre-training. In *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, 1035–1047.
- [90] Zongjie Li, Chaozheng Wang, Zhibo Liu, Haoxuan Wang, Dong Chen, Shuai Wang, and Cuiyun Gao. 2023. CCTEST: Testing and repairing code completion systems. In *45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 1238–1250.
- [91] Feng Lin, Dong Jae Kim, and Tse-Husn (Peter) Chen. 2024. When LLM-based code generation meets the software development process. arXiv:2403.15852. Retrieved from <https://arxiv.org/abs/2403.15852>
- [92] Jinfeng Lin, Yalin Liu, Qingkai Zeng, Meng Jiang, and Jane Cleland-Huang. 2021. Traceability transformed: Generating more accurate links with pre-trained Bert models. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 324–335.
- [93] Yishuai Lin, Philippe Descamps, Nicolas Gaud, Vincent Hilaire, and Abderrafaa Koukam. 2015. Multi-agent system for intelligent Scrum project management. *Integrated Computer-Aided Engineering* 22, 3 (2015), 281–296.
- [94] Richard C. Linger, Harlan D. Mills, and Bernard I. Witt. 1979. *Structured Programming - Theory and Practice*. Addison-Wesley.
- [95] Shuo Liu, Jacky Keung, Zhen Yang, Fang Liu, Qilin Zhou, and Yihan Liao. 2024. Delving into parameter-efficient fine-tuning in code change learning: An empirical study. In *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 465–476.
- [96] Zhe Liu, Chunyang Chen, Junjie Wang, Xing Che, Yuekai Huang, Jun Hu, and Qing Wang. 2023. Fill in the blank: Context-aware automated text input generation for mobile GUI testing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1355–1367.
- [97] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Zhilin Tian, Yuekai Huang, Jun Hu, and Qing Wang. 2024. Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model. In *IEEE/ACM 46th International Conference on Software Engineering*, 1–12.
- [98] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. StarCoder 2 and the Stack v2: The next generation. arXiv:2402.19173. Retrieved from <https://arxiv.org/abs/2402.19173>
- [99] Guilong Lu, Xiaolin Ju, Xiang Chen, Wenlong Pei, and Zhilong Cai. 2024. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. *Journal of Systems and Software* 212 (2024), 112031.
- [100] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. LLaMA-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 647–658.
- [101] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021*.
- [102] Sebastian Lubos, Alexander Felfernig, Thi Ngoc Trang Tran, Damian Garber, Merfat El Mansi, Seda Polat Erdeniz, and Viet-Man Le. 2024. Leveraging LLMs for the quality assurance of software requirements. In *32nd IEEE International Requirements Engineering Conference (RE '24)*. Grischa Liebel, Irit Hadar, and Paola Spoletini (Eds.), IEEE, 389–397.



- [103] Xianchang Luo, Yinxing Xue, Zhenchang Xing, and Jiamou Sun. 2022. PRCBERT: Prompt learning for requirement classification using Bert-based pretrained language models. In *37th IEEE/ACM International Conference on Automated Software Engineering*, 1–13.
- [104] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. WizardCoder: Empowering code large language models with Evol-Instruct. arXiv:2306.08568. Retrieved from <https://arxiv.org/abs/2306.08568>
- [105] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2024. SpecGen: Automated generation of formal program specifications via large language models. arXiv:2401.08807. Retrieved from <https://arxiv.org/abs/2401.08807>
- [106] Lipeng Ma, Weidong Yang, Bo Xu, Sihang Jiang, Ben Fei, Jiaqing Liang, Mingjie Zhou, and Yanghua Xiao. 2024. KnowLog: Knowledge enhanced pre-trained language model for log understanding. In *46th IEEE/ACM International Conference on Software Engineering (ICSE '24)*. ACM, 32:1–32:13.
- [107] Ambarish Moharil and Arpit Sharma. 2022. Identification of intra-domain ambiguity using transformer-based machine learning. In *1st International Workshop on Natural Language-based Software Engineering*, 51–58.
- [108] Ambarish Moharil and Arpit Sharma. 2023. Tabasco: A transformer based contextualization toolkit. *Science of Computer Programming* 230 (2023), 102994.
- [109] Fangwen Mu, Xiao Chen, Lin Shi, Song Wang, and Qing Wang. 2023. Developer-intent driven code comment generation. In *45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 768–780.
- [110] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-based prompt selection for code-related few-shot learning. In *45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 2450–2462.
- [111] Phuong T. Nguyen, Juri Di Rocco, Claudio Di Sipio, Riccardo Rubel, Davide Di Ruscio, and Massimiliano Di Penta. 2024. GPTSniffer: A CodeBERT-based classifier to detect source code written by ChatGPT. *Journal of Systems and Software* 214 (2024), 112059.
- [112] Chao Ni, Xin Yin, Kaiwen Yang, Dehai Zhao, Zhenchang Xing, and Xin Xia. 2023. Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation. arXiv:2308.11237. Retrieved from <https://arxiv.org/abs/2308.11237>
- [113] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. CodeGen2: Lessons for training LLMs on programming and natural languages. arXiv:2305.02309. Retrieved from <https://arxiv.org/2305.02309>
- [114] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An open large language model for code with multi-turn program synthesis. In *the 11th International Conference on Learning Representations (ICLR '23)*. OpenReview.net.
- [115] Bashar Nuseibeh and Steve M. Easterbrook. 2000. Requirements engineering: A roadmap. In *22nd International Conference on Software Engineering, Future of Software Engineering Track (ICSE '00)*. ACM, 35–46.
- [116] Shengyi Pan, Lingfeng Bao, Jiayuan Zhou, Xing Hu, Xin Xia, and Shanping Li. 2024. Towards more practical automation of vulnerability assessment. In *IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 148, 13 pages. DOI : <https://doi.org/10.1145/3597503.3639110>
- [117] Shengyi Pan, Jiayuan Zhou, Filipe Roseiro Cogo, Xin Xia, Lingfeng Bao, Xing Hu, Shanping Li, and Ahmed E. Hassan. 2022. Automated unearthing of dangerous issue reports. In *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, New York, NY, 834–846. DOI : <https://doi.org/10.1145/3540250.3549156>
- [118] Eugenio Parra, Jose Luis de la Vara, and Luis Alonso. 2018. Analysis of requirements quality evolution. In *40th International Conference on Software Engineering: Companion Proceedings (ICSE '18)*. ACM, 199–200.
- [119] Ricardo Silva Peres, Xiaodong Jia, Jay Lee, Keyi Sun, Armando Walter Colombo, and Jose Barata. 2020. Industrial artificial intelligence in industry 4.0-systematic review, challenges and outlook. *IEEE Access* 8 (2020), 220121–220139.
- [120] Laura Plein, Wendkûuni C. Ouedraogo, Jacques Klein, and Tegawendé F. Bissyandé. 2024. Automatic generation of test cases based on bug reports: A feasibility study with large language models. In *2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 360–361.
- [121] Chanathip Pornprasit and Chakkrit Tantithamthavorn. 2024. GPT-3.5 for code review automation: How do few-shot learning, prompt design, and model fine-tuning impact their performance? arXiv:2402.00905. Retrieved from <https://arxiv.org/abs/2402.00905>
- [122] Amrit Poudel, Jinfeng Lin, and Jane Cleland-Huang. 2023. Leveraging transformer-based language models to automate requirements satisfaction assessment. arXiv:2312.04463. Retrieved from <https://arxiv.org/abs/2312.04463>
- [123] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J. Hellendoorn. 2023. CAT-LM training language models on aligned code and tests. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 409–420.
- [124] Juri Di Rocco, Davide Di Ruscio, Claudio Di Sipio, Phuong T. Nguyen, and Riccardo Rubel. 2024. On the use of large language models in model-driven engineering. arXiv:2410.17370. Retrieved from <https://arxiv.org/abs/2410.17370>
- [125] Krishna Ronanki, Beatriz Cabrero-Daniel, and Christian Berger. 2022. ChatGPT as a tool for user story quality evaluation: Trustworthy out of the box? In *International Conference on Agile Software Development*. Springer, 173–181.

- [126] W. W. Royce. 1987. Managing the development of large software systems: Concepts and techniques. In *9th International Conference on Software Engineering*. ACM Press, 328–339.
- [127] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open foundation models for code. arXiv:2308.12950. Retrieved from <https://arxiv.org/abs/2308.12950>
- [128] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Illic, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. 2022. BLOOM: A 176B-parameter open-access multilingual language model. arXiv:2211.05100. Retrieved from <https://arxiv.org/abs/2211.05100>
- [129] Bo Shen, Jiaxin Zhang, Taihong Chen, Daoguang Zan, Bing Geng, An Fu, Muhan Zeng, Ailun Yu, Jichuan Ji, Jingyang Zhao, et al. 2023. PanGu-Coder2: Boosting large language models for code with ranking feedback. arXiv:2307.14936. Retrieved from <https://arxiv.org/abs/2307.14936>
- [130] Jiho Shin, Sepehr Hashtroudi, Hadi Hemmati, and Song Wang. 2024. Domain adaptation for code model-based unit test case generation. In *33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 1211–1222.
- [131] Seung Yeob Shin, Fabrizio Pastore, Domenico Bianculli, and Alexandra Baicoianu. 2024. Towards generating executable metamorphic relations using large language models. In *International Conference on the Quality of Information and Communications Technology*. Springer, 126–141.
- [132] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2024. Using large language models to generate JUnit tests: An empirical study. In *28th International Conference on Evaluation and Assessment in Software Engineering*, 313–322.
- [133] Mehmet Söylemez, Bedir Tekinerdogan, and Ayça Kolukisa Tarhan. 2022. Challenges and solution directions of microservice architectures: A systematic literature review. *Applied Sciences* 12, 11 (2022), 5507.
- [134] Giriprasad Sridhara, Ranjani H. G., and Sourav Mazumdar. 2023. ChatGPT: A study on its utility for ubiquitous software engineering tasks. arXiv:2305.16837. Retrieved from <https://arxiv.org/abs/2305.16837>
- [135] Benjamin Steenhoeck, Michele Tufano, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. Reinforcement learning from automatic feedback for high-quality unit test generation. arXiv:2310.02368. Retrieved from <https://arxiv.org/abs/2310.02368>
- [136] Robert Boyce Stone. 1997. *Towards a Theory of Modular Design*. The University of Texas at Austin.
- [137] Zhihong Sun, Chen Lyu, Bolun Li, Yao Wan, Hongyu Zhang, Ge Li, and Zhi Jin. 2024. Enhancing code generation performance of smaller models by distilling the reasoning ability of LLMs. arXiv:2403.13271. Retrieved from <https://arxiv.org/abs/2403.13271>
- [138] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. Intellicode compose: Code generation using transformer. In *28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1433–1443.
- [139] Sumanth Tatineni. 2023. Applying DevOps practices for quality and reliability improvement in cloud-based systems. *Technix International Journal for Engineering Research* 10, 11 (2023), 374–380.
- [140] Christopher Thompson and David A. Wagner. 2017. A large-scale study of modern code review and security in open source projects. In *PROMISE*. ACM, 83–92.
- [141] Patanamon Thongtanunam, Chanathip Pornprasit, and Chakkrit Tantithamthavorn. 2022. AutoTransform: Automated code transformation to support modern code review process. In *44th International Conference on Software Engineering*, 237–248.
- [142] Zhao Tian, Junjie Chen, and Xiangyu Zhang. 2023. Test-case-driven programming understanding in large language models for better code generation. arXiv:2309.16120. Retrieved from <https://arxiv.org/abs/2309.16120>
- [143] Christos Tsigkanos, Pooja Rani, Sebastian Müller, and Timo Kehrer. 2023. Large language models: The next frontier for variable discovery within metamorphic testing? In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 678–682.
- [144] Rosalia Tufano. 2023. Automating code review. In *ICSE Companion Proceedings*. IEEE, 192–196.
- [145] Rosalia Tufano, Ozren Dabić, Antonio Mastropaolo, Matteo Ciniselli, and Gabriele Bavota. 2024. Code review automation: Strengths and weaknesses of the state of the art. *IEEE Transactions on Software Engineering* 50, 2 (2024), 338–353.
- [146] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using pre-trained models to boost code review automation. In *44th International Conference on Software Engineering*, 2291–2302.
- [147] Rosalia Tufano, Luca Pascarella, Michele Tufano, Denys Poshyvanyk, and Gabriele Bavota. 2021. Towards automating code review activities. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 163–174.
- [148] Chaozheng Wang, Junhao Hu, Cuiyun Gao, Yu Jin, Tao Xie, Hailiang Huang, Zhenyu Lei, and Yuetang Deng. 2023. How practitioners expect code completion? In *31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.), ACM, 1294–1306.

- [149] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R. Lyu. 2022. No more fine-tuning? An experimental evaluation of prompt tuning in code intelligence. In *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, 382–394.
- [150] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R. Lyu. 2023. Prompt tuning in code intelligence: An experimental evaluation. *IEEE Transactions on Software Engineering* 49, 11 (2023), 4869–4885.
- [151] Chong Wang, Jian Zhang, Yebo Feng, Tianlin Li, Weisong Sun, Yang Liu, and Xin Peng. 2024. Teaching code LLMs to use autocompletion tools in repository-level code generation. arXiv:2401.06391. Retrieved from <https://arxiv.org/abs/2401.06391>
- [152] Deze Wang, Boxing Chen, Shanshan Li, Wei Luo, Shaoliang Peng, Wei Dong, and Xiangke Liao. 2023. One adapter for all programming languages? Adapter tuning for code search and summarization. In *International Conference on Software Engineering (ICSE)*. IEEE, 5–16.
- [153] Jiexin Wang, Xitong Luo, Liuwen Cao, Hongkui He, Hailin Huang, Jiayuan Xie, Adam Jatowt, and Yi Cai. 2024. Is your AI-generated code really secure? Evaluating large language models on secure code generation with CodeSecEval. arXiv:2407.02395. Retrieved from <https://arxiv.org/abs/2407.02395>
- [154] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven Hoi. 2023. CodeT5+: Open code large language models for code understanding and generation. In *the 2023 Conference on Empirical Methods in Natural Language Processing*.
- [155] Yawen Wang, Lin Shi, Mingyang Li, Qing Wang, and Yun Yang. 2020. A deep context-wise method for coreference detection in natural language requirements. In *2020 IEEE 28th International Requirements Engineering Conference (RE)*. IEEE, 180–191.
- [156] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *2021 Conference on Empirical Methods in Natural Language Processing (EMNLP '21)*. Association for Computational Linguistics, 8696–8708.
- [157] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2023. RCA-gent: Cloud root cause analysis by autonomous agents with tool-augmented large language models. arXiv:2310.16340. Retrieved from <https://arxiv.org/abs/2310.16340>
- [158] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 24824–24837.
- [159] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2023. Magicoder: Source code is all you need. arXiv:2312.02120. Retrieved from <https://arxiv.org/abs/2312.02120>
- [160] Cheng Wen, Yuandao Cai, Bin Zhang, Jie Su, Zhiwu Xu, Dugang Liu, Shengchao Qin, Zhong Ming, and Tian Cong. 2024. Automatically inspecting thousands of static bug warnings with large language model: How far are we? *ACM Transactions on Knowledge Discovery from Data* 18, 7 (2024), 1–34.
- [161] Xin-Cheng Wen, Cuiyun Gao, Jiaxin Ye, Yichen Li, Zhihong Tian, Yan Jia, and Xuan Wang. 2024. Meta-path based attentional graph learning model for vulnerability detection. *IEEE Transactions on Software Engineering* 50, 3 (2024), 360–375.
- [162] Xin-Cheng Wen, Xincheng Wang, Yujia Chen, Ruida Hu, David Lo, and Cuiyun Gao. 2024. VulEval: Towards repository-level evaluation of software vulnerability detection. arXiv:2404.15596. Retrieved from <https://arxiv.org/abs/2404.15596>
- [163] Xin-Cheng Wen, Yupan Chen, Cuiyun Gao, Hongyu Zhang, Jie M. Zhang, and Qing Liao. 2023. Vulnerability detection with graph simplification and enhanced graph representation learning. In *45th International Conference on Software Engineering (ICSE '23)*. IEEE Press, 2275–2286. DOI: <https://doi.org/10.1109/ICSE48619.2023.00191>
- [164] Xin-Cheng Wen, Cuiyun Gao, Shuzheng Gao, Yang Xiao, and Michael R. Lyu. 2024. SCALE: Constructing structured natural language comment trees for software vulnerability detection. In *33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*. ACM, New York, NY, 235–247. DOI: <https://doi.org/10.1145/3650212.3652124>
- [165] Xin-Cheng Wen, Xincheng Wang, Cuiyun Gao, Shaohua Wang, Yang Liu, and Zhaoquan Gu. 2024. When less is enough: Positive and unlabeled learning model for vulnerability detection. In *38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE Press, 345–357. DOI: <https://doi.org/10.1109/ASE56229.2023.00144>
- [166] Jules White, Sam Hays, Quchen Fu, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. arXiv:2303.07839. Retrieved from <https://arxiv.org/abs/2303.07839>
- [167] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4all: Universal fuzzing with large language models. In *IEEE/ACM 46th International Conference on Software Engineering*, 1–13.
- [168] Danning Xie, Byungwoo Yoo, Nan Jiang, Mijung Kim, Lin Tan, Xiangyu Zhang, and Judy S. Lee. 2023. Impact of large language models on generating software specifications. arXiv:2306.03324. Retrieved from <https://arxiv.org/abs/2306.03324>

- [169] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. 2023. ChatUniTest: A ChatGPT-based automated unit test generation tool. arXiv:2305.04764. Retrieved from <https://arxiv.org/abs/2305.04764>
- [170] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. WizardLM: Empowering large language models to follow complex instructions. arXiv:2304.12244. Retrieved from <https://arxiv.org/abs/2304.12244>
- [171] Junjielong Xu, Ziang Cui, Yuan Zhao, Xu Zhang, Shilin He, Pinjia He, Liqun Li, Yu Kang, Qingwei Lin, Yingnong Dang, et al. 2024. UniLog: Automatic logging via LLM and in-context learning. In *46th IEEE/ACM International Conference on Software Engineering (ICSE '24)*. ACM, 14:1–14:12.
- [172] Hao Yan, Thomas D. Latoza, and Ziyu Yao. 2024. IntelliExplain: Enhancing interactive code generation through natural language explanations for non-professional programmers. arXiv:2405.10250. Retrieved from <https://arxiv.org/abs/2405.10250>
- [173] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 technical report. arXiv:2407.10671. Retrieved from <https://arxiv.org/abs/2407.10671>
- [174] Hui Yang, Alistair Willis, Anne N. De Roeck, and Bashar Nuseibeh. 2010. Automatic detection of noxious coordination ambiguities in natural language requirements. In *25th IEEE/ACM International Conference on Automated Software Engineering (ASE '10)*. ACM, 53–62.
- [175] Zezhou Yang, Cuiyun Gao, Zhaoqiang Guo, Zhenhao Li, Kui Liu, Xin Xia, and Yuming Zhou. 2024. A survey on modern code review: Progresses, challenges and opportunities. arXiv:2405.18216. Retrieved from <https://arxiv.org/abs/2405.18216>
- [176] Stephen S. Yau and Jeffrey J. P. Tsai. 1986. A survey of software design techniques. *IEEE Transactions on Software Engineering* 12, 6 (1986), 713–721.
- [177] Juyeon Yoon, Robert Feldt, and Shin Yoo. 2023. Autonomous large language model agents enabling intent-driven mobile GUI testing. arXiv:2311.08649. Retrieved from <https://arxiv.org/abs/2311.08649>
- [178] Shengcheng Yu, Chunrong Fang, Yuchen Ling, Chentian Wu, and Zhenyu Chen. 2023. LLM for test script generation and migration: Challenges, capabilities, and opportunities. In *23rd IEEE International Conference on Software Quality, Reliability, and Security (QRS '23)*. IEEE, 206–217.
- [179] Zhaojian Yu, Xin Zhang, Ning Shang, Yangyu Huang, Can Xu, Yishujie Zhao, Wenxiang Hu, and Qiufeng Yin. 2023. Wavocoder: Widespread and versatile enhanced instruction tuning with refined data generation. arXiv:2312.14187. Retrieved from <https://arxiv.org/abs/2312.14187>
- [180] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No more manual tests? Evaluating and improving ChatGPT for unit test generation. arXiv:2305.04207. Retrieved from <https://arxiv.org/abs/2305.04207>
- [181] Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Yongji Wang, and Jian-Guang Lou. 2023. Large language models meet NL2Code: A survey. In *61st Annual Meeting of the Association for Computational Linguistics (ACL '23)*. Association for Computational Linguistics, 7443–7464.
- [182] Dylan Zhang, Xuchao Zhang, Chetan Bansal, Pedro Henrique B. Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2023. PACE-LM: Prompting and augmentation for calibrated confidence estimation with GPT-4 in cloud incident root cause analysis. arXiv:2309.05833. Retrieved from <https://arxiv.org/abs/2309.05833>
- [183] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. arXiv:2303.12570. Retrieved from <https://arxiv.org/abs/2303.12570>
- [184] Fengji Zhang, Zexian Zhang, Jacky Wai Keung, Xiangru Tang, Zhen Yang, Xiao Yu, and Wenhua Hu. 2024. Data preparation for deep learning based code smell detection: A systematic literature review. *Journal of Systems and Software* 216 (2024), 112131.
- [185] Junwei Zhang, Zhongxin Liu, Xing Hu, Xin Xia, and Shanping Li. 2023. Vulnerability detection by learning from syntax-based execution paths of code. *IEEE Transactions on Software Engineering* 49, 8 (2023), 4196–4212.
- [186] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. CodeAgent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. arXiv:2401.07339. Retrieved from <https://arxiv.org/abs/2401.07339>
- [187] Kechi Zhang, Huangzhao Zhang, Ge Li, Jia Li, Zhuo Li, and Zhi Jin. 2023. Toolcoder: Teach code generation models to use API search tools. arXiv:2305.04032. Retrieved from <https://arxiv.org/abs/2305.04032>
- [188] Renrui Zhang, Jiaming Han, Chris Liu, Aojun Zhou, Pan Lu, Yu Qiao, Hongsheng Li, and Peng Gao. 2024. LLaMA-adaptor: Efficient fine-tuning of large language models with zero-initialized attention. In *International Conference on Learning Representations (ICLR)*. OpenReview.net.
- [189] Simiao Zhang, Jiaping Wang, Guoliang Dong, Jun Sun, Yueling Zhang, and Geguang Pu. 2024. Experimenting a new programming practice with LLMs. arXiv:2401.01062. Retrieved from <https://arxiv.org/abs/2401.01062>

- [190] Yifan Zhang, Dave Towey, and Matthew Pike. 2023. Automated metamorphic-relation generation with ChatGPT: An experience report. In *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 1780–1785.
- [191] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. 2023. CodeGeeX: A pre-trained model for code generation with multilingual benchmarking on HumanEval-X. In *29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*. ACM, 5673–5684.
- [192] Jiayuan Zhou, Michael Pacheco, Jinfu Chen, Xing Hu, Xin Xia, David Lo, and Ahmed E. Hassan. 2023. CoLeFunDa: Explainable silent vulnerability fix identification. In *45th International Conference on Software Engineering (ICSE '23)*. IEEE Press, 2565–2577. DOI: <https://doi.org/10.1109/ICSE48619.2023.00214>
- [193] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, Junda He, and David Lo. 2023. Generation-based code review automation: How far are we? In *2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*. IEEE, 215–226.

Received 5 April 2024; revised 18 December 2024; accepted 18 December 2024